

Motif-Preserving Graph Clustering on Continuous-Time Temporal Graphs

Longlong Lin
Southwest University
longlonglin@swu.edu.cn

Jin Zhao
Huazhong University
of Science and
Technology
zjin@hust.edu.cn

Hongchao Qin
Beijing Institute of
Technology
qhc.neu@gmail.com

Qiangqiang Dai
Beijing Institute of
Technology
qiangd66@gmail.com

Zeli Wang
Chongqing University
of Posts and
Telecommunications
zlwang@cqupt.edu.cn

ABSTRACT

Temporal graphs naturally model complex real-life dynamic systems, including financial, social, and transportation networks. While temporal graph clustering has achieved significant success, neither continuous topological-temporal dependencies nor higher-order connectivity patterns have been well-explored, leading to sub-optimal performance. To alleviate these dilemmas, we introduce a novel problem called *Motif-preserving graph clustering on Continuous-time Temporal graphs (MgcCT)*. Namely, given a continuous-time temporal graph and a temporal motif, *MgcCT* aims to detect a high-quality cluster by preserving the connectivity of higher-order structures built up from the temporal motif. To this end, we first propose a new concept of temporal motif conductance, which can be interpreted as the probability of escape from a random walk in the cluster. Then, we formulate *MgcCT* as an optimization task that aims at finding a cluster with minimum temporal motif conductance. Subsequently, we prove that the *MgcCT* problem is NP-hard and propose two efficient algorithms. Specifically, we first develop an approximate global algorithm *AGA* and theoretically prove *AGA* can achieve the quadratic Cheeger bound. To further boost efficiency, we then propose a heuristic local algorithm *HLA* without accessing the entire graph. Finally, extensive empirical results on eight real-world datasets showcase the effectiveness, efficiency, and scalability of our solutions compared to ten competitors.

PVLDB Reference Format:

Longlong Lin, Jin Zhao, Hongchao Qin, Qiangqiang Dai, and Zeli Wang. Motif-Preserving Graph Clustering on Continuous-Time Temporal Graphs. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Graph clustering is a fundamental graph data mining task, which has been widely studied in image segmentation [64, 69], load balancing [12], and machine learning [7, 39]. However, real-life graphs often contain complex but useful time information (i.e., the edges are associated with rich timestamps), which provides insights into the underlying mechanisms [27, 28]. For example, in collaboration networks, the timestamps of the edges indicate when the corresponding authors co-authored papers. Therefore, many researchers are working on designing various temporal graph

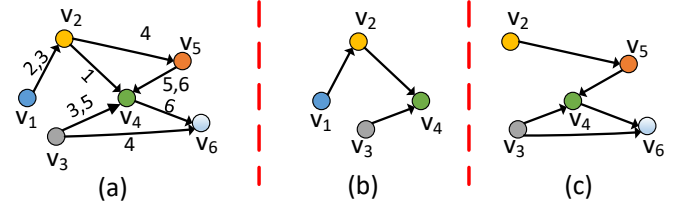


Figure 1: (a) is the continuous-time temporal graph. (b) and (c) are time-aggregated snapshots by aggregating timestamps 1-3 and 4-6.

clustering algorithms to simultaneously consider the entanglement of structure and temporal dynamics [11, 54, 57, 65].

Despite the significant success of temporal graph clustering, existing methods suffer from two-fold defects. Firstly, they approximately compress the raw continuous-time temporal networks to the discrete time-aggregated snapshots, in which each snapshot consists of edges that occur in the user-specified temporal resolution (e.g., one hour or day). Unfortunately, the discrete and non-continuous temporal graphs seriously affect the effectiveness of approaches [9, 24, 56, 57]. Imagine that when the resolution is too coarse, it may not distinguish paths correctly, which is an important indicator for clustering [16, 36, 52]. Taking Fig. 1 as an example, (v_1, v_2, v_4) of (b) is an invalid path with respect to (w.r.t.) time-order in (a), and the valid path (v_1, v_2, v_5) of (a) cannot be identified by (b) and (c). Besides, it may also break some important temporal structures, such as burst events that occur over a short period of time. Therefore, these existing methods overlook important continuous topological-temporal dependencies and may identify sub-optimal results, as stated in our empirical results (Section 5). Secondly, they adopt the dyadic temporal edge as the atomic clustering unit to partition the graph, while significantly ignoring the frequently occurring higher-order temporal connectivity patterns (a.k.a., temporal motifs [49, 58]). Actually, unlike temporal edges, each temporal motif is a small subgraph with temporal constraints (e.g., temporal ordering and duration) that indicates some unique functions [33, 48, 72]. Therefore, using such mesoscopic-level temporal motifs as the new clustering unit (i.e., each temporal motif should be regarded as a whole that cannot be split [58]) for temporal graph clustering has both theoretical significance and practical value but remains nascent. To name but a few, in temporal money transfer networks, the cycle structure that follows a chronological order with short duration (named temporal cycle) is an important factor in judging money laundering [34, 37, 66]. Thus, the clusters with dense temporal cycles can be used to identify financial criminal gangs. In temporal customer-product networks, the temporal butterfly (i.e., the (2,2)-biclique with temporal constraints) is a significant pattern of buying behavior [8]. Therefore, the clusters with dense temporal butterflies are suitable to enhance the accuracy and effectiveness of recommendations. Summing up, detecting motif-preserving clusters on continuous-time temporal networks can reveal some interesting phenomena, which has been verified in our experiments (Section 5).

Motif-preserving graph clustering (a.k.a., higher-order graph clustering) can typically be addressed by the following three potential models [41, 43]: motif modularity, motif density, and motif conductance. The first two models only consider the internal higher-order cohesiveness and overlook the important external sparsity, which is an important factor in measuring the clustering quality [25, 31, 36, 40, 50], resulting in unsatisfactory empirical performance (Section 5). This aspect, however, is addressed by the third model [5, 6, 30, 70, 74, 76, 81]. Inspired by this, we move beyond traditional motif conductance by integrating temporal information and formulating a novel concept of temporal motif conductance. Specifically, temporal motif conductance is the ratio of the total number of temporal motif instances going out of the cluster to the total number of temporal motif instances within the cluster (Definition 6). Smaller temporal motif conductance implies the partition preserves rich temporal motif instances inside the cluster and cuts temporal motif instances as little as possible. Besides, temporal motif conductance can be viewed as the escape probability of the cluster (Theorem 2.1), which generalizes the random walk interpretation of conventional conductance [53, 74]. Based on this, we formulate our problem of Motif-preserving graph clustering on Continuous-time Temporal graphs (*MgcCT*) as an optimization task that aims at finding a cluster with the minimum temporal motif conductance. Consequently, from the perspectives of both cut and escape probability, *MgcCT* has strong interpretability.

MgcCT is practically important but technically challenging. For example, one simple treatment for solving *MgcCT* is to utilize static higher-order graph clustering methods [6, 30, 70] to temporal networks by ignoring the time information of edges, but such solutions greatly compress the informative time semantics behind temporal graphs and thus obtain sub-optimal clustering qualities (Section 5). Another possible method is to project the original temporal graph into a static weighted graph by enumerating the temporal motif and then adopting the classic graph clustering algorithm to partition the weighted graph. However, such a naive solution has only provable clustering qualities for the temporal motif consisting of three vertices [6], which limits real-world applications [49, 51, 68, 78]. Besides, it requires an exact enumeration of all temporal motif instances, which incurs prohibitively high time&space overheads (is even NP-hard for the simple temporal star [48]) due to the topological-temporal combinatorial explosions [51, 58, 68, 78]. On the other hand, as delineated in Section 6, many temporal lower-order graph clustering algorithms have been proposed in the literature, encompassing modularity-based [54], conductance-based [65], and density-based [57] methods. However, these approaches overlook the more insightful higher-order substructures, ultimately leading to subpar empirical clustering performance (Section 5). Thus, existing methods suffer from sub-optimal clustering qualities, limited applications, and entail prohibitive computational costs.

To overcome these challenges, we first propose an approximate global algorithm *AGA* based on our proposed *edge-focused* weighted graph projection method. Specifically, we observe that the main reason why existing methods cannot obtain an accuracy guarantee is that the temporal motif cut (Definition 3) does not have a theoretical guarantee in the corresponding weighted graph (Section 3). The explanation behind this phenomenon is that during the construction of the weighted graph, the weights of edges are obtained by enumerating hyperedges (i.e., temporal motifs), resulting in the loss or magnification of the original edge information. To recover this information, *AGA* specifies that an edge’s weight is non-zero iff it also exists as an edge in the original temporal graph (Definition 9). We further prove that this simple yet important adjustment

achieves the quadratic Cheeger bound, which is our main theoretical contribution. Additionally, unlike existing methods that exhaustively enumerate all temporal motif instances, *AGA* leverages highly efficient combinatorial counting techniques to directly compute the edge weights, significantly reducing computational costs. To further boost the efficiency, we devise a heuristic local algorithm, *HLA*, which does not require accessing the entire graph. Specifically, *HLA* starts a random walk from a query node to obtain a *local* probability distribution π (i.e., π has very few non-zero elements) by dynamically computing the edge weights on the fly. This approach greatly reduces the time and space overheads by avoiding the need to enumerate unnecessary temporal instances in advance. Then, *HLA* executes a *sweep-cut* procedure over π to obtain a heuristic result. Our main contributions are summarized as follows.

- **A Novel Model.** We propose a novel problem of Motif-preserving graph clustering on Continuous-time Temporal graphs (*MgcCT*) by introducing temporal motif conductance, a higher-order temporal clustering objective that jointly captures internal motif preservation and external motif separability. We further show that temporal motif conductance admits an exact cluster-conditioned one-step escape-probability interpretation, which provides a complementary random-walk view of the proposed objective.
- **Two Efficient Algorithms.** We prove the NP-hardness of our problem and propose two efficient algorithms. Specifically, we first devise an approximate global algorithm *AGA* based on a new edge-focused weighted graph projection, leading to a Cheeger-style quadratic approximation guarantee. Then, a heuristic pure local algorithm *HLA* is proposed, which computes edge-focused weights on demand during local diffusion, striking a better trade-off between efficiency and quality.
- **Extensive Experiments.** Empirical results on eight real-world datasets demonstrate our proposed solutions indeed are efficient and effective when contrasted with ten competitors. Notably, our algorithms outperform the baselines by 1~4 orders of magnitude in speed with higher clustering quality, alongside a notable reduction in memory usage of up to an order of magnitude. The source code of this paper can be found at [3].

2 PRELIMINARIES

2.1 Basic Notations

Let $\mathcal{G}(V, \mathcal{E})$ be a directed continuous-time temporal graph, where V denotes the node set and $\mathcal{E}$ represents the directed temporal edge set. Each temporal edge $(u, v, t) \in \mathcal{E}$ indicates an interaction from node u to node v at a specific timestamp $t \in \mathbb{R}$. Note that (u, v, t_i) and (u, v, t_j) are regarded as two different temporal edges if $t_i \neq t_j$. Namely, nodes u and v can be connected at multiple different timestamps. Let $n = |V|$ and $m = |\mathcal{E}|$ be the number of nodes and temporal edges, respectively. For any node subset $S \subseteq V$, we denote $\mathcal{G}_S(S, \mathcal{E}_S)$ is a temporal subgraph of $\mathcal{G}(V, \mathcal{E})$ iff $\mathcal{E}_S = \{(u, v, t) | u, v \in S\} \subseteq \mathcal{E}$. Compared with previous studies that aggregate temporal edges into discrete snapshots [44–46, 54, 65, 79, 82], the continuous-time temporal graph considered in this paper does not perform any timestamp aggregation on the raw datasets. Thus, in this paper, we examine more general and purely temporal graphs that can preserve fine-grained temporal dynamics in a lossless fashion [9, 24, 56, 57]. For convenience, we use the temporal graph to represent the directed continuous-time temporal graph when the context is clear. We use $G(V, E)$ to denote the de-temporal graph of $\mathcal{G}$, in which $E = \{(u, v) | (u, v, t) \in \mathcal{E}\}$ and $|E| = \bar{m}$. That is, G is a simple static directed graph obtained by discarding timestamps and merging duplicate directed edges of $\mathcal{G}$. Fig. 2(a) shows a de-temporal graph G of Fig. 1(a). Although there are many temporal motif definitions, we use the most representative

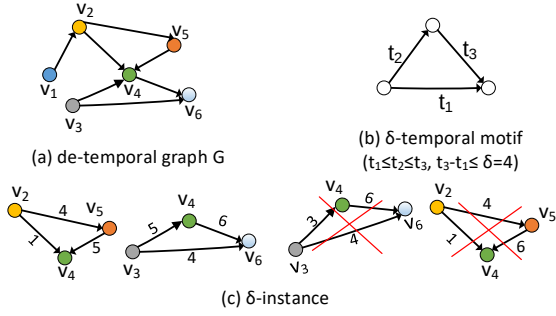


Figure 2: The latter two of (c) are not δ -instances because they respectively violate conditions (i) and (ii) in Definition 2.

δ -temporal motif [48, 58, 61, 62, 72] due to its generality, but our results can be easily extended to others [49]¹.

DEFINITION 1. [δ -temporal motif [58]] A δ -temporal motif M_δ is a temporal graph that can be represented as an edge stream $M_\delta = \{(x_1, y_1, t_1), (x_2, y_2, t_2), \dots, (x_l, y_l, t_l)\}$ where $t_i \leq t_{i+1}$ for all $1 \leq i \leq l-1$, and $t_l - t_1 \leq \delta$.

In this paper, unless otherwise specified, a δ -temporal motif requires that its de-temporal graph be weakly connected [58].

DEFINITION 2. [δ -instance [58]] The temporal subgraph $\mathcal{G}_S = \{(u_1, v_1, t'_1), (u_2, v_2, t'_2), \dots, (u_l, v_l, t'_l)\}$, where $t'_i \leq t'_{i+1}$ for all $1 \leq i' \leq l-1$, is called a δ -instance of M_δ , denoted as $\mathcal{G}_S \in M_\delta$, if the following conditions are satisfied.

(i) **Temporal ordering:** There exists a bijection h on the nodes such that $h(x_i) = u_i$ and $h(y_i) = v_i$ for $i = 1, 2, \dots, l$;

(ii) **Temporal duration:** The time differences of the temporal edges in $\mathcal{G}_S$ do not exceed δ , i.e., $t'_l - t'_1 \leq \delta$.

Fig. 2(b) and (c) show a toy δ -temporal motif and the corresponding δ -instances, respectively.

2.2 Problem Statement

Motif-preserving graph clustering has been extensively studied in static networks, which aims to identify a cluster C such that the vertices within C participate in many motif instances and few motif instances cross C and $\bar{C} = V \setminus C$ [5, 6, 30, 70, 74, 76, 81]. However, existing methods ignore the frequently appearing temporal properties of graphs, resulting in sub-optimal performance (details in Section 5). Inspired by this, we propose a new concept of temporal motif conductance, generalizing the motif conductance by integrating temporal information.

DEFINITION 3. [δ -temporal motif cut] Given a node subset C and a δ -temporal motif M_δ , the temporal motif cut of C is denoted by $\text{cut}_{M_\delta}(C) = \sum_{\mathcal{G}_S \in M_\delta} I(S \cap C \neq \emptyset \& S \cap \bar{C} \neq \emptyset)$. Where $I(\cdot)$ is the indicator function.

DEFINITION 4. [δ -temporal motif degree] Given a δ -temporal motif M_δ and a node u , the temporal motif degree of node u is denoted by $d_{M_\delta}(u) = \sum_{\mathcal{G}_S \in M_\delta} I(u \in S)$.

DEFINITION 5. [δ -temporal motif volume] Given a node subset C and a δ -temporal motif M_δ , the temporal motif volume of C is denoted by $\text{vol}_{M_\delta}(C) = \sum_{u \in C} d_{M_\delta}(u)$.

¹Since our models and algorithms only depend on temporal motif instances, other temporal motif definitions can be directly extended to our solutions by the corresponding enumeration or counting algorithms.

DEFINITION 6. [δ -temporal motif conductance] Given a node set C and a δ -temporal motif M_δ , the temporal motif conductance of C is denoted by $\phi_{M_\delta}(C) = \frac{\text{cut}_{M_\delta}(C)}{\min\{\text{vol}_{M_\delta}(C), \text{vol}_{M_\delta}(\bar{C})\}}$. For completeness, if the denominator is zero, we set $\phi_{M_\delta}(C) = +\infty$.

Problem formulation. Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif M_δ , the goal of the Motif-preserving graph clustering on Continuous-time Temporal graphs (MgcCT) is to identify a vertex subset C^* such that $\phi_{M_\delta}(C^*) \leq \phi_{M_\delta}(C)$ for any $C \subseteq V$. We use $\phi_{M_\delta}^*$ to represent $\phi_{M_\delta}(C^*)$.

According to Definition 6, we know that C^* is the sparsest balanced cluster, in which the sparsest signifies to minimize the δ -instances that cross C^* and $V \setminus C^*$, and the balanced indicates the δ -instances that are in the cluster. Thus, minimizing temporal motif conductance can find a cluster that preserves seamlessly higher-order temporal connectivity.

The advantages of MgcCT. (1) It provides a new clustering metric based on higher-order temporal motifs rather than simple temporal edges, allowing users to flexibly focus the clusters that highlight specific temporal motifs relevant to a given type of temporal network. For instance, temporal cycles are crucial features of financial markets, and emphasizing these clusters with dense temporal cycles can provide strong interpretability and improve the clustering quality. (2) It also admits an exact cluster-conditioned one-step escape-probability interpretation (Theorem 2.1), offering a complementary probabilistic perspective on why a cluster with small temporal motif conductance tends to preserve temporal motif instances internally while cutting few such instances across the boundary.

2.3 Problem Analysis

This subsection enhances our problem through the lens of escape probability. Then, we prove the problem's NP-hardness.

DEFINITION 7. [δ -cluster-constrained temporal motif walk] Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal motif M_δ with k nodes, and a cluster $C \subseteq V$, we define a one-step cluster-constrained temporal motif walk as follows. Given the current node u : (1) If $d_{M_\delta}(u) = 0$, the walker stays at u . (2) Otherwise, the walker first samples a δ -instance $\mathcal{G}_S \in M_\delta$ containing u with probability $\Pr(\mathcal{G}_S | u) = \frac{I(u \in S)}{d_{M_\delta}(u)}$. (3) If $S \subseteq C$ or $S \subseteq \bar{C}$, the walker stays at u . (4) Otherwise, let $i = |S \cap C|$ with $1 \leq i \leq k-1$. If $u \in C$, then with probability $\frac{1}{i}$, the walker moves to a node chosen uniformly at random from $S \cap \bar{C}$; with the remaining probability $1 - \frac{1}{i}$, it stays at u . If $u \in \bar{C}$, then with probability $\frac{1}{k-i}$, the walker moves to a node chosen uniformly at random from $S \cap C$; with the remaining probability $1 - \frac{1}{k-i}$, it stays at u .

Remark. The factors $\frac{1}{i}$ and $\frac{1}{k-i}$ are crucial. For a crossing δ -instance $\mathcal{G}_S$ with $|S \cap C| = i$, the same instance is counted once for each of its i nodes in C and once for each of its $k-i$ nodes in $\bar{C}$. Dividing by these multiplicities ensures that each crossing δ -instance contributes exactly once to the total escape probability. Therefore, the following theorem gives an exact probabilistic interpretation of temporal motif conductance.

THEOREM 2.1. [δ -escape probability] Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal motif M_δ with k nodes, and a cluster C . Consider the cluster-constrained temporal motif walk in Definition 7. If the initial state w_0 is sampled from C (or $\bar{C}$) according

to the temporal motif degree distribution, we have $\phi_{M_\delta}(C) = \max \{ \Pr(w_1 \in \bar{C} \mid w_0 \in C), \Pr(w_1 \in C \mid w_0 \in \bar{C}) \}$.

PROOF. We first prove the case of escaping from C to $\bar{C}$. By Definition 7, if the walk starts from a node $u \in C$, then the probability of moving to $\bar{C}$ in one step is

$\Pr(w_1 \in \bar{C} \mid w_0 = u) = \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i \text{ and } u \in S)}{d_{M_\delta}(u)}$. Therefore, we have $\Pr(w_1 \in \bar{C} \mid w_0 \in C) = \sum_{u \in C} \Pr(w_1 \in \bar{C} \mid w_0 = u) \Pr(w_0 = u \mid w_0 \in C)$

$$= \sum_{u \in C} \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i \text{ and } u \in S)}{d_{M_\delta}(u)} \frac{d_{M_\delta}(u)}{\text{vol}_{M_\delta}(C)} = \frac{\sum_{u \in C} \sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i \text{ and } u \in S)}{\text{vol}_{M_\delta}(C)} = \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i) \sum_{u \in C} I(u \in S)}{\text{vol}_{M_\delta}(C)} = \frac{\sum_{i=1}^{k-1} \frac{1}{i} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i) \cdot i}{\text{vol}_{M_\delta}(C)} = \frac{\sum_{i=1}^{k-1} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(|S \cap C| = i)}{\text{vol}_{M_\delta}(C)} = \frac{\sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(S \cap C \neq \emptyset \text{ and } S \cap \bar{C} \neq \emptyset)}{\text{vol}_{M_\delta}(C)} = \frac{\text{cut}_{M_\delta}(C)}{\text{vol}_{M_\delta}(C)}.$$

Analogously, for the walk starting from $\bar{C}$, we have $\Pr(w_1 \in C \mid w_0 \in \bar{C}) = \frac{\text{cut}_{M_\delta}(C)}{\text{vol}_{M_\delta}(\bar{C})}$.

Finally, according to Definition 6, we have $\phi_{M_\delta}(C) = \frac{\text{cut}_{M_\delta}(C)}{\min\{\text{vol}_{M_\delta}(C), \text{vol}_{M_\delta}(\bar{C})\}} = \max \left\{ \frac{\text{cut}_{M_\delta}(C)}{\text{vol}_{M_\delta}(C)}, \frac{\text{cut}_{M_\delta}(C)}{\text{vol}_{M_\delta}(\bar{C})} \right\}$. Combining the above equalities completes the proof. $\square$

By Theorem 2.1, temporal motif conductance admits an exact one-step escape-probability interpretation. Concretely, for a cluster C , its temporal motif conductance equals the larger of the two probabilities that the above walk moves from C to $\bar{C}$ or from $\bar{C}$ to C in one step.

THEOREM 2.2. [Hardness] Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif M_δ , the problem of identifying $MgcCT$ from $\mathcal{G}(V, \mathcal{E})$ is NP-hard.

PROOF. To prove that $MgcCT$ is NP-hard, we reduce the minimum motif conductance problem to our $MgcCT$ problem. Specifically, (1) *Instance Construction*: Let $G(V, E)$ be a de-temporal (i.e., static) graph and M be a de-temporal (i.e., static) motif. The minimum motif conductance problem is defined as finding a cluster $C \subseteq V$ that minimizes $\phi_M(C) = \frac{\text{cut}_M(C)}{\min(\text{vol}_M(C), \text{vol}_M(\bar{C}))}$, where $\text{cut}_M(C)$ denotes the number of M -instances crossing the boundary between C and $\bar{C}$, and $\text{vol}_M(C)$ denotes the total number of M -instances involving nodes in C . (2) *Reduction to $MgcCT$* : Construct a temporal graph $\mathcal{G}(V, \mathcal{E})$ from $G(V, E)$ by assigning all edges the same timestamp t_0 . Define a δ -temporal motif M_δ as the temporal extension of M with $\delta = 0$, ensuring that M_δ -instances in $\mathcal{G}$ are exactly equivalent to M -instances in G . (3) *Equivalence Verification*: For any cluster $C \subseteq V$, the temporal motif conductance of C in $\mathcal{G}$ (Definition 6) satisfies: $\phi_{M_\delta}(C) = \frac{\text{cut}_{M_\delta}(C)}{\min(\text{vol}_{M_\delta}(C), \text{vol}_{M_\delta}(\bar{C}))} = \phi_M(C)$. Thus, finding the cluster C^* that minimizes $\phi_{M_\delta}(C)$ (the $MgcCT$ problem) is equivalent to finding the cluster C^* that minimizes $\phi_M(C)$ (the minimum motif conductance problem). (4) *Conclusion*: Since the minimum motif conductance problem is NP-hard [6], and the above reduction can be completed in polynomial time, the $MgcCT$ problem is also NP-hard. $\square$

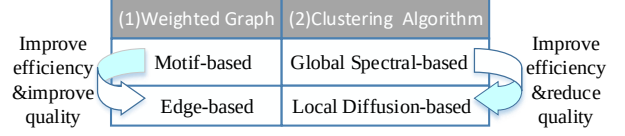


Figure 3: Key functions of the seminal two-stage framework.

3 EXISTING SOLUTIONS

According to Theorem 2.2, the intrinsic complexity of the proposed problem necessitates the development of approximate algorithms to efficiently optimize our objectives. At a high level, most existing methods adhere to the well-known two-stage clustering framework [6, 41, 70, 76]. As shown in Figure 3, in the first stage, the input graph is transformed into a static weighted graph, where the weight of each edge corresponds to the number of motif instances in which the edge participates. In the second stage, the static weighted graph is partitioned using any graph clustering algorithm. Different algorithms differentiate themselves in how they efficiently and effectively partition the static weighted graph. For instance, Benson et al. [6] and Yin et al. [76] employ the Fiedler vector-based global spectral clustering algorithm and the local diffusion graph clustering algorithm, respectively, to partition the static weighted graph. Building on this idea, we introduce the following key concepts. Let $\mathcal{A} \in \mathbb{R}^{n \times n}$ denote the δ -temporal motif weighted graph (matrix), where $\mathcal{A}_{uv} = \mathcal{A}_{vu} = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} I(u \in S \wedge v \in S)$ for all $u, v \in V$. It is crucial to note that u and v may not necessarily be connected by an edge in the original de-temporal graph (i.e., $(u, v) \notin E$). Using these notations, several basic existing solutions are outlined below (details in their original papers).

3.1 Order-Based Solutions

These solutions can be summarized in two steps: (1) Sort all vertices according to a predefined rule. Without loss of generality, let the sorted vertices be denoted as $u_1, u_2, \dots, u_n$. (2) Output $S = \arg \min \phi_{\mathcal{A}}(S_i)$, where $S_i = \{u_1, u_2, \dots, u_i\}$ and $\phi_{\mathcal{A}}(S_i)$ represents the traditional edge-based conductance [42, 71] of S_i on the weighted graph $\mathcal{A}$. Specifically, $\phi_{\mathcal{A}}(S_i) = \frac{\sum_{u \in S_i, v \notin S_i} \mathcal{A}_{uv}}{\min(\sum_{u \in S_i} \mathcal{D}_{uu}, \sum_{v \notin S_i} \mathcal{D}_{vv})}$, where $\mathcal{D}_{uu} = \sum_v \mathcal{A}_{uv}$. Consequently, the following two order-based solutions are proposed: *THSC* [6] and *THLC* [76]. In particular, *THSC* [6] and *THLC* [76] utilize the Fiedler Vector and Personalized PageRank vector, respectively, to sort all vertices. Note that [80] and *THLC* [76] are conceptually similar: both approaches begin by enumerating temporal motif instances and subsequently perform local graph diffusion. The main distinction is that [80] implements the diffusion process via the tensor formed by all motif instances. Therefore, [80] remains largely of theoretical interest, as it is challenging to implement in practice and suffers from poor and highly unstable empirical performance, largely due to the presence of numerous hard-to-tune parameters [41, 43]. Given that *THLC* [76] has already demonstrated superior empirical results over [80], we omit further experimental comparisons, as they would be redundant. For convenience, Table 1 summarizes these basic solutions.

Limitations of Order-Based Solutions. From Table 1, we observe the following: (1) Although *THSC* has successfully established the Cheeger inequality (i.e., quadratic bound) for motifs with three vertices, its applicability to motifs containing four or more vertices remains unproven. This limitation is significant because larger motifs are often more practical and widely used in various applications [49, 51, 68, 78]. (2) *THLC* is a heuristic algorithm, and

Table 1: A comparison of *MgcCT* algorithms. k is the number of nodes of the given temporal motif. Note that $k = 2$ is not discussed, because in this situation, our model degenerates into the traditional lower-order temporal graph clustering and not the focus of this paper. $\times$ represents the corresponding method has no accuracy guarantee. $\phi_{\mathcal{M}_\delta}^* \in (0, 1)$ is the smallest temporal motif conductance. $O(E(\mathcal{M}_\delta))$ is the time complexity of enumeration temporal motif instances and $O(C(\mathcal{M}_\delta))$ is the time complexity of counting all edge-focused weights. $O(E(\mathcal{M}_\delta))$ is much slower than $O(C(\mathcal{M}_\delta))$ [38, 51, 58, 68, 78]. $num(\mathcal{M}_\delta)$ is the number of temporal motif instances. $\Delta(u)$ is the time cost of calculating the edge-focused weighted degree of node u and $\Delta_{max} = \max\{\Delta(u)|u \in V\}$. α and ϵ are parameters less than 1.

Methods	Accuracy Guarantee	Time Complexity	Space Complexity	Weighted Graph
<i>THSC</i> [6]	$O(\sqrt{\phi_{\mathcal{M}_\delta}^*})$ for $k = 3$; $\times$ for $k > 3$	$O(E(\mathcal{M}_\delta))$	$O(\min\{n^2, \binom{k}{2} num(\mathcal{M}_\delta)\})$	Motif-based
<i>THLC</i> [76]	$\times$	$O(E(\mathcal{M}_\delta))$	$O(\min\{n^2, \binom{k}{2} num(\mathcal{M}_\delta)\})$	Motif-based
<i>THPR</i> [41]	$O(1/2 + 1/2\phi_{\mathcal{M}_\delta}^*)$ for any fixed motif	$O(E(\mathcal{M}_\delta))$	$O(m)$	Motif-based
<i>AGA</i> (Our)	$O(\sqrt{\phi_{\mathcal{M}_\delta}^*})$ for any fixed motif	$O(C(\mathcal{M}_\delta))$	$O(m)$	Edge-based
<i>HLA</i> (Our)	$\times$	$O(\frac{\Delta_{max}}{\alpha\epsilon})$	$O(\frac{1}{\alpha\epsilon})$	Edge-based

its clustering quality is significantly influenced by numerous hard-to-tune parameters and seeding strategies. In practical applications, its performance can be unstable and may even yield very poor solutions, as demonstrated by our experimental results. (3) *THSC* and *THLC* suffer from significant computational and memory inefficiencies due to their need to enumerate and store all temporal motif instances.

3.2 The Peeling-based Solution

Recently, Lin et al. [41] presented a peeling-based method that seamlessly integrates the computation and partitioning of $\mathcal{A}$ in an iterative fashion, yielding results that come with approximation guarantees. Specifically, they redefine the concept of motif conductance, where temporal motif conductance represents a specialized instance when the motif is a δ -temporal motif, and introduce a peeling-based greedy algorithm that iteratively refines motif conductance by initiating the optimization process from each individual vertex. To align with the requirements of our problem, we provide the following definitions.

DEFINITION 8. (*Temporal Motif Resident*) For a vertex set C , the temporal motif resident of a vertex $u \in C$ with respect to C is defined as: $Tr_C(u) = \frac{d_{\mathcal{M}_\delta}(u) + d_{\mathcal{M}_\delta}(u, C) - d_{\mathcal{M}_\delta}(u, \bar{C})}{d_{\mathcal{M}_\delta}(u)}$, in which $d_{\mathcal{M}_\delta}(u, H) = \sum_{S \in \mathcal{M}_\delta} I(u \in S, S - \{u\} \subseteq H)$ and H can be either C or $\bar{C}$.

By Definition 8, the Temporal Higher-order Peeling Removal (*THPR*) algorithm is outlined as follows: First, it calculates the temporal motif resident for each vertex. Subsequently, it iteratively eliminates the vertex with the smallest temporal motif. Finally, it outputs the result that exhibits the smallest temporal motif conductance during the iteration procedure.

Limitations of the Peeling-Based Approach. As indicated in Table 1, although *THPR* offers results with approximation guarantees, our empirical evaluation reveals that it exhibits suboptimal practical clustering quality. More importantly, *THPR* imposes excessively high time costs due to the necessity of enumerating all temporal motif instances, which leads to a combinatorial explosion and diminished efficiency.

4 THE PROPOSED SOLUTIONS FOR *MGCCT*

This section first constructs a novel weighted graph to approximate temporal motif cut and temporal motif volume, and then proposes a global algorithm with theoretical approximation guarantees. Finally, we present a heuristic local algorithm for scalability.

4.1 Novel Edge-focused Weighted Graph

By Table 1, it is evident that *THSC* is theoretically well-justified when $k = 3$. Nevertheless, in the more general scenario where $k > 3$, *THSC* fails to deliver results accompanied by provable approximation guarantees. The primary impediment lies in the ambiguity surrounding the correlation between the temporal motif cut $cut_{\mathcal{M}_\delta}(C)$ and the weighted graph cut $\sum_{u \in C, v \notin C} \mathcal{A}_{uv}$, a topic extensively discussed in [6]. Specifically, the question of whether $cut_{\mathcal{M}_\delta}(C)$ can be constrained by $\sum_{u \in C, v \notin C} \mathcal{A}_{uv}$ remains unresolved [6]. The underlying rationale for this limitation is that the existing reweighting clustering approach initially constructs a hypergraph with hyperedges representing all temporal motif instances. Subsequently, it converts this hypergraph into a weighted graph, wherein the weight of an edge (u, v) is indicative of the number of times the vertex pair $\{u, v\}$ is involved in the hyperedges. This transformation process, however, leads to the forfeiture of the original edge information. Consequently, a pivotal inquiry arises: Can we devise a novel weighted graph projection method that facilitates the analysis of the relationship between $cut_{\mathcal{M}_\delta}(C)$ and $\sum_{u \in C, v \notin C} \mathcal{A}_{uv}$, while simultaneously allowing for the examination of the association between the temporal motif volume $vol_{\mathcal{M}_\delta}(C)$ and weighted graph volume $\sum_{u \in C} \mathcal{D}_{uu}$? In this subsection, we offer an affirmative response to this question by introducing an innovative edge-focused weighted graph projection method that retains the original edge information.

Before delving into the formal definition, we briefly highlight the mathematical intuition that underpins the efficiency of our approach. Existing higher-order clustering methods inherently rely on motif enumeration to explicitly materialize every temporal motif instance as a hyperedge [6, 41, 76]. However, evaluating temporal motif cuts and volumes ultimately requires only the aggregate number of motif instances each original edge is involved in (Lemmas 4.1 and 4.2). By shifting the paradigm from exact enumeration to combinatorial counting, the following edge-focused projection computes these aggregate weights directly. This fundamental shift bypasses the prohibitive memory and computation costs associated with combinatorial explosions, as we no longer need to construct or store the explicit node-sets of individual motif instances.

DEFINITION 9. [*edge-focused weighted matrix*] Given a temporal graph $\mathcal{G}$ with its de-temporal graph $G = (V, E)$ and a δ -temporal motif $\mathcal{M}_\delta$, the edge-focused weighted matrix $W_{ef} \in \mathbb{R}^{n \times n}$ is defined:

$$(W_{ef})_{uv} = \begin{cases} \sum_{S \in \mathcal{M}_\delta} I((u, v) \in E_S^{\delta t}), & (u, v) \in E \\ 0, & (u, v) \notin E \end{cases} \quad (1)$$

Where E_S^{dt} denotes the directed de-temporal edge set of a temporal motif instance $\mathcal{G}_S \in \mathcal{M}_\delta$.

Remark. By Definition 9, it is apparent that the distinctive feature of our edge-focused weighted matrix (graph) in comparison to existing motif-based weighted graphs is that it exclusively assigns weights to edges that are present in the original graph. Specifically, if an edge (u, v) does not exist in E , its edge-focused weight is set to zero. In essence, an edge in the edge-focused weighted graph (matrix) possesses a non-zero weight if and only if it also appears as an edge in the original de-temporal graph G and participates in at least one temporal motif instance. We underscore that, despite its simplicity, this modification engenders several substantial benefits: (1) We can achieve high-quality clustering results with provable approximation guarantees (Theorem 4.3), a novel theoretical advancement not found in existing literature. (2) In the worst-case scenario, the size of the edge-focused weighted graph W_{ef} is $O(\bar{m})$, which is more compact than the motif-based weighted graph $\mathcal{A}$, whose size is $O(\min\{n^2, \binom{k}{2} \text{num}(\mathcal{M}_\delta)\})$, where $\text{num}(\mathcal{M}_\delta)$ represents the number of motif instances. Consequently, partitioning W_{ef} requires less time and space compared to $\mathcal{A}$. (3) The edge-focused weighted matrix can be directly derived using combinatorial counting techniques, eliminating the need for exhaustive enumeration of all temporal motif instances, thereby significantly reducing computational overhead.

Let $H_\delta = (V_\delta, E_\delta)$ be the directed de-temporal graph of $\mathcal{M}_\delta$, where $|E_\delta| = m_\delta$. For any nonempty proper subset $X \subset V_\delta$, define $\partial_{H_\delta}(X) = \{(a, b) \in E_\delta \mid a \in X, b \notin X \text{ or } a \notin X, b \in X\}$. That is, $\partial_{H_\delta}(X)$ denotes the set of directed de-temporal motif edges crossing the cut $(X, V_\delta \setminus X)$, regardless of edge directions. We define $\underline{c}_\delta = \min_{\emptyset \neq X \subset V_\delta} |\partial_{H_\delta}(X)|$, $\bar{c}_\delta = \max_{\emptyset \neq X \subset V_\delta} |\partial_{H_\delta}(X)|$. Similarly, for each motif vertex $x \in V_\delta$, define its incident de-temporal degree as $d_{H_\delta}(x) = |\{(x, y) \in E_\delta\}| + |\{(y, x) \in E_\delta\}|$. Let $\underline{d}_\delta = \min_{x \in V_\delta} d_{H_\delta}(x)$, $\bar{d}_\delta = \max_{x \in V_\delta} d_{H_\delta}(x)$. The following pivotal lemmas that illuminate the correlation between $\text{cut}_{\mathcal{M}_\delta}(\cdot)$ and $\text{vol}_{\mathcal{M}_\delta}(\cdot)$ within the original temporal graph $\mathcal{G}$ and our proposed edge-focused weighted graph W_{ef} , which establish a solid theoretical foundation that underpins the development of our subsequent algorithms.

LEMMA 4.1. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a cluster $C \subseteq V$, and a δ -temporal motif $\mathcal{M}_\delta$. Let $W = W_{ef} + W_{ef}^\top$, we have*

$$\underline{c}_\delta \text{cut}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C, v \notin C} W_{uv} \leq \bar{c}_\delta \text{cut}_{\mathcal{M}_\delta}(C).$$

PROOF. For any temporal motif instance $\mathcal{G}_S \in \mathcal{M}_\delta$, we let $H_S = (S, E_S^{dt})$ be its directed de-temporal graph. Define $b_C(\mathcal{G}_S) = |\{(x, y) \in E_S^{dt} \mid x \in S \cap C, y \in S \cap \bar{C} \text{ or } x \in S \cap \bar{C}, y \in S \cap C\}|$. Namely, $b_C(\mathcal{G}_S)$ is the number of directed de-temporal edges of $\mathcal{G}_S$ whose endpoints are separated by the cut $(C, \bar{C})$, regardless of edge directions. By Definition 9 and $W = W_{ef} + W_{ef}^\top$, we have $\sum_{u \in C, v \notin C} W_{uv} = \sum_{u \in C, v \notin C} ((W_{ef})_{uv} + (W_{ef})_{vu}) = \sum_{u \in C, v \notin C} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} (\mathbb{I}((u, v) \in E_S^{dt}) + \mathbb{I}((v, u) \in E_S^{dt})) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} b_C(\mathcal{G}_S)$. If $\mathcal{G}_S$ is not cut by C , namely $S \cap C = \emptyset$ or $S \cap \bar{C} = \emptyset$, then $b_C(\mathcal{G}_S) = 0$. Otherwise, the cut $(S \cap C, S \cap \bar{C})$ induces a nonempty proper partition of the motif vertices under the isomorphism between $\mathcal{M}_\delta$ and $\mathcal{G}_S$. Therefore, by the definitions of $\underline{c}_\delta$ and $\bar{c}_\delta$, we have $\underline{c}_\delta \leq b_C(\mathcal{G}_S) \leq \bar{c}_\delta$. Consequently, $\underline{c}_\delta \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} \mathbb{I}(S \cap C \neq \emptyset, S \cap \bar{C} \neq \emptyset) \leq \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} b_C(\mathcal{G}_S) \leq \bar{c}_\delta \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} \mathbb{I}(S \cap C \neq \emptyset, S \cap \bar{C} \neq \emptyset)$. By

Algorithm 1 Approximate Global Algorithm (AGA)

Input: A temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$
Output: A cluster C

```

1:  $G \leftarrow$  the de-temporal graph of  $\mathcal{G}$ 
2: Initializing  $W_{ef}$  and  $\mathcal{D}$  are zero matrices
3: for each edge  $(u, v) \in G$  do
4:    $(W_{ef})_{uv} \leftarrow$  the edge-focused weight by Equation 1
5:  $W \leftarrow W_{ef} + W_{ef}^\top$ 
6: for each  $u \in V$  do
7:    $\mathcal{D}_{uu} \leftarrow \sum_{v \in V} W_{uv}$ 
8:  $\mathcal{L} \leftarrow I - \mathcal{D}^{-1/2} W \mathcal{D}^{-1/2}$ 
9:  $z \leftarrow$  eigenvector of the second smallest eigenvalue of  $\mathcal{L}$ 
10:  $z_i \leftarrow$  the index of  $z$  with  $i$ th smallest value
11:  $C \leftarrow \arg \min \phi_W(C_i)$ , where  $C_i = \{z_1, z_2, \dots, z_i\}$  and  $\phi_W(C_i)$  is the edge-
    based conductance of  $C_i$  in  $W$ 
12: return  $C$ 
```

Definition 3, the summation on both sides is exactly $\text{cut}_{\mathcal{M}_\delta}(C)$. Thus, $\underline{c}_\delta \text{cut}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C, v \notin C} W_{uv} \leq \bar{c}_\delta \text{cut}_{\mathcal{M}_\delta}(C)$, which completes the proof. $\square$

LEMMA 4.2. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a cluster $C \subseteq V$, and a δ -temporal motif $\mathcal{M}_\delta$. Let $W = W_{ef} + W_{ef}^\top$, we have*

$$\underline{d}_\delta \text{vol}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C} \sum_{v \in V} W_{uv} \leq \bar{d}_\delta \text{vol}_{\mathcal{M}_\delta}(C).$$

PROOF. For a temporal motif instance $\mathcal{G}_S \in \mathcal{M}_\delta$ and a vertex $u \in V$, define $a_u(\mathcal{G}_S) = |\{(u, v) \in E_S^{dt} \mid v \in S\}| + |\{(v, u) \in E_S^{dt} \mid v \in S\}|$. That is, $a_u(\mathcal{G}_S)$ is the number of directed de-temporal edges of $\mathcal{G}_S$ incident to u , regardless of edge directions. By Definition 9 and $W = W_{ef} + W_{ef}^\top$, for any vertex $u \in V$, we have $\sum_{v \in V} W_{uv} = \sum_{v \in V} ((W_{ef})_{uv} + (W_{ef})_{vu}) = \sum_{v \in V} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} (\mathbb{I}((u, v) \in E_S^{dt}) + \mathbb{I}((v, u) \in E_S^{dt})) = \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} a_u(\mathcal{G}_S)$. If $u \notin S$, then $a_u(\mathcal{G}_S) = 0$. If $u \in S$, then u corresponds to some motif vertex $x \in V_\delta$ under the isomorphism between $\mathcal{M}_\delta$ and $\mathcal{G}_S$. Hence, $a_u(\mathcal{G}_S) = d_{H_\delta}(x)$. By the definitions of $\underline{d}_\delta$ and $\bar{d}_\delta$, we have $\underline{d}_\delta \leq a_u(\mathcal{G}_S) \leq \bar{d}_\delta$ whenever $u \in S$. Equivalently, $\underline{d}_\delta \mathbb{I}(u \in S) \leq a_u(\mathcal{G}_S) \leq \bar{d}_\delta \mathbb{I}(u \in S)$. Summing over all $\mathcal{G}_S \in \mathcal{M}_\delta$ and all $u \in C$, we obtain $\underline{d}_\delta \text{vol}_{\mathcal{M}_\delta}(C) = \underline{d}_\delta \sum_{u \in C} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} \mathbb{I}(u \in S) \leq \sum_{u \in C} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} a_u(\mathcal{G}_S) = \sum_{u \in C} \sum_{v \in V} W_{uv} \leq \bar{d}_\delta \sum_{u \in C} \sum_{\mathcal{G}_S \in \mathcal{M}_\delta} \mathbb{I}(u \in S) = \bar{d}_\delta \text{vol}_{\mathcal{M}_\delta}(C)$. This completes the proof. $\square$

Implications of Lemma 4.1 and Lemma 4.2. Lemma 4.1 and Lemma 4.2 show that the weighted graph $W = W_{ef} + W_{ef}^\top$ preserves both the temporal motif cut and the temporal motif volume up to motif-dependent constants. Specifically, the cut approximation is controlled by the motif cut-crossing constants $\underline{c}_\delta$ and $\bar{c}_\delta$, while the volume approximation is controlled by the motif incident-degree constants $\underline{d}_\delta$ and $\bar{d}_\delta$. As a result, when spectral clustering is applied to W , it offers provable approximation guarantees (Theorem 4.3). However, existing motif-based methods have inaccurate estimates for them, resulting in subpar quality (see Table 1 for details).

4.2 AGA: The Approximate Global Algorithm

Based on the preceding analyses, we propose an approximate global algorithm, denoted as AGA, which is detailed in Algorithm 1. Specifically, Algorithm 1 initiates by computing the edge-focused weighted matrix W_{ef} (Lines 1-4). Subsequently, we construct a symmetric weighted matrix W by $W \leftarrow W_{ef} + W_{ef}^\top$ (Line 5) and

derive the diagonal degree matrix $\mathcal{D}$ with $\mathcal{D}_{uu} = \sum_{v \in V} W_{uv}$ (Lines 6-7). Then, Algorithm 1 derives the normalized Laplacian matrix $\mathcal{L}$ by $\mathcal{L} \leftarrow I - \mathcal{D}^{-1/2} W \mathcal{D}^{-1/2}$ (Line 8). Following this, Algorithm 1 proceeds to identify the eigenvector z that corresponds to the second smallest eigenvalue of $\mathcal{L}$, facilitating the execution of the sweep-cut procedure (Lines 9-11). This process entails arranging all entries of z in increasing order, ensuring $z_1 \leq z_2 \leq \dots \leq z_n$, and then selecting the set $C = \arg \min \phi_W(C_i)$, where $C_i = \{z_1, z_2, \dots, z_i\}$ and $\phi_W(C_i)$ denotes the conventional edge-based conductance [42, 71] of C_i on W . Namely, $\phi_W(C_i) = \frac{\sum_{u \in C_i, v \notin C_i} W_{uv}}{\min\{\sum_{u \in C_i} \sum_{v \in V} W_{uv}, \sum_{u \notin C_i} \sum_{v \in V} W_{uv}\}}$. Finally, Algorithm 1 outputs C as the target cluster (Line 12).

Remark. The symmetrization step in Line 5 is introduced only to enable the use of the standard normalized Laplacian and the classical Cheeger inequality [1] (see Theorem 4.2) for undirected weighted graphs. It does *not* convert the original temporal graph or the query temporal motif into an undirected one. More specifically, all temporal motif instances are still matched on the original directed continuous-time temporal graph according to Definitions 1 and 2, where both temporal ordering and edge directions are preserved. The directed edge-focused weight $(W_{ef})_{uv}$ is computed before symmetrization and counts only directed temporal motif instances in which the directed edge (u, v) participates. The symmetrized matrix $W = W_{ef} + W_{ef}^T$ should therefore be understood as a final clustering projection rather than a redefinition of the temporal motif semantics. It aggregates the two directed edge-focused contributions between a pair of vertices only for spectral partitioning. This design provides a clean Cheeger-style analysis while preserving directional and temporal constraints during the weight construction stage.

The proofs of the following theorems are put in the full paper [3] due to space limitations.

THEOREM 4.1. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$, the time and space complexity of Algorithm 1 are $O(C(\mathcal{M}_\delta))$ and $O(m)$, respectively. $O(C(\mathcal{M}_\delta))$ is the time complexity of counting all edge-focused weights.*

Remark. By Table 1, it is clear that existing methods depend on temporal motif enumeration algorithms to obtain the hyperedges (i.e., temporal motif instances) and then construct the motif-based weighted graph for clustering. In stark contrast, our novel AGA algorithm is fundamentally based on the temporal motif counting method, which can significantly reduce the time complexity. Furthermore, as stated in Theorem 4.1, AGA necessitates only linear space complexity for the storage of basic data structures. This is unlike other algorithms (except for THPR [41]), which typically need to store all temporal motif instances, resulting in high space complexity. Consequently, our proposed AGA algorithm surpasses SOTA methods in terms of efficiency, as stated in our experiments.

THEOREM 4.2. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$. Let λ_2 be the second smallest eigenvalue of $\mathcal{L}$ (Line 8), we have $\frac{d_\delta}{2c_\delta} \lambda_2 \leq \phi_{\mathcal{M}_\delta}^* \leq \frac{\bar{d}_\delta}{c_\delta} \sqrt{2\lambda_2}$.*

THEOREM 4.3. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$ and a δ -temporal motif $\mathcal{M}_\delta$. Let C be the cluster returned by Algorithm 1, we have $\phi_{\mathcal{M}_\delta}(C) \leq 2 \frac{\bar{d}_\delta}{c_\delta} \sqrt{\frac{\bar{c}_\delta}{d_\delta} \phi_{\mathcal{M}_\delta}^*}$. In particular, when the temporal motif is fixed, Algorithm 1 achieves a Cheeger-style quadratic approximation guarantee $\phi_{\mathcal{M}_\delta}(C) = O(\sqrt{\phi_{\mathcal{M}_\delta}^*})$.*

Remark. According to Table 1, our proposed algorithm AGA attains an asymptotically equivalent level of clustering quality when compared to the THSC method. Notably, while THSC cannot furnish quality assurances when $k > 3$, our AGA guarantees uniform quality performance across all values of k . In contrast, evaluating the clustering quality of AGA against THPR presents a more intricate analysis, as the performance is contingent upon $\phi_{\mathcal{M}_\delta}^*$. Nevertheless, as evidenced by the empirical findings outlined in Section 5, AGA consistently outperforms THPR, a phenomenon that can be attributed to the fact that THPR is designed with a generic graph focus, thereby neglecting the unique temporal dynamics inherent in temporal graphs. Thus, our proposed AGA method yields clusters that are not only theoretically grounded in high-quality guarantees but also empirically validated to surpass SOTA approaches.

4.3 HLA: The Heuristic Local Algorithm

Although our global approximate algorithm AGA is substantially faster than existing enumeration-based methods, it still needs to precompute $(W_{ef})_{uv}$ for every edge (u, v) in the de-temporal graph, and hence must access the entire graph before returning the target cluster. On massive temporal graphs, such a global procedure can still be expensive for interactive applications. For example, on the SO dataset with nearly 50 million temporal edges, AGA still requires hours to process one query (Section 5).

Fortunately, on most real-world graphs, the target clusters are often localized, meaning that the proportion of the vertices belonging to the target cluster is very small [6, 41, 43, 76]. As an empirical observation (shown in Table 2), the clusters returned by AGA under several representative motifs are small relative to the entire graph. In particular, the ratio of the number of vertices in the target cluster comprises no more than 5% of the total vertices in the original graph. Besides, in numerous real-world scenarios, users are often willing to tolerate a certain degree of inaccuracy in exchange for faster response times, especially in large networks. Therefore, beyond improving the asymptotic complexity of global clustering, it is also desirable to design a *pure local* algorithm whose cost depends mainly on the explored region rather than the entire graph, striking a better balance between efficiency and quality.

A natural idea is to adopt the well-known local graph diffusion framework. However, existing local higher-order clustering methods are not purely local in our setting. For instance, [76] still requires globally enumerating temporal motif instances and materializing a weighted graph as an index before the graph diffusion phase. Consequently, its query processing is local only after paying a substantial global preprocessing cost. In contrast, we seek a method that performs *no global motif enumeration, no weighted-graph materialization, and no external indexing*. To this end, we propose a heuristic local algorithm, denoted by HLA, that computes edge-focused weights *on demand* during the diffusion process. The key novelty of HLA is not merely replacing the weight definition in a standard local diffusion routine. Instead, HLA tightly couples local propagation with online evaluation of the edge-focused weighted graph. Specifically, when a vertex becomes active during the diffusion process, HLA computes its edge-focused weighted degree and the edge-focused weights of its incident edges only at that moment. These locally computed quantities are then reused, avoiding repeated computation while never requiring the full weighted graph to be constructed. As a result, HLA only touches a small fraction of the graph around the query node, which makes it a genuinely *pure local, index-free, and on-demand* algorithm.

Before introducing HLA, we briefly review the well-known Push subroutine [2], which is used to approximate the Personalized

Table 2: The ratio of the number of vertices in the identified cluster to that of the whole graph.

Dataset	# vertices	M_1	M_2	M_3
CM	1,893	46(2.43%)	15(0.79%)	95(5.01%)
UB	152,599	12(0.0078%)	55(0.036%)	14(0.0091%)
SU	189,191	108(0.057%)	403(0.21%)	6(0.0032%)
SO	2,572,345	41(0.0015%)	14(0.00054%)	49(0.0019%)

PageRank vector. The *Push* algorithm initiates its operation from a source state (node) s and proceeds to disseminate information locally. Throughout its execution, the algorithm iteratively refines two key variables associated with each state u : its reserve $\pi(u)$ and its residue $r(u)$. Specifically, $\pi(u)$ serves as an approximation of the Personalized PageRank value of state u with respect to the source state s , while $r(u)$ quantifies the information that remains to be propagated from state u to other states. In each iteration (Lines 19-24 of Algorithm 2), the *Push* algorithm identifies states u that require information dissemination (Line 20). For each such state u , the algorithm transfers a portion $\alpha r(u)$ to the reserve $\pi(u)$ (Line 24), while the remaining $(1 - \alpha)r(u)$ is distributed among the neighboring states in proportion to the edge weights connecting them, rather than being evenly distributed (Lines 21-22). After finishing the propagation, the algorithm resets the residue $r(u)$ to zero (Line 24). Following [2, 76], ϵ is used to balance quality and efficiency.

Based on these backgrounds, *HLA* is described in Algorithm 2. In Line 1, it initializes the residue $r(s) = 1$ for the source node s and implicitly sets $r(u) = 0$ for all $u \neq s$, while the reserve $\pi(u)$ is initialized as zero for all $u \in V$. In implementation, r and π are maintained as sparse vectors, where unstored entries are implicitly treated as zero. This avoids any global initialization cost and preserves the locality of *HLA*. In Line 2, it initializes the queue $Q = \{s\}$ as the nodes to be pushed, and the set $D = \emptyset$ as the nodes whose weight information has already been calculated to avoid recomputing the edge-focused weighted degree of a vertex. In Lines 3-8, it locally calculates edge-focused weighted degree and edge weights on demand to perform the push operation (Line 8, Lines 19-24) until the queue Q becomes empty (Line 3). In particular, Line 4 pops a vertex u from the queue Q and then determines whether the edge-focused weighted degree of u has been calculated. If not (Line 5), it adds u into set D (Line 6) and executes Line 7 to obtain the edge-focused weighted degree of u and the edge-focused weight of its neighbor (Lines 14-18). Importantly, this computation is performed *only when u is first activated*, rather than for all nodes in advance. After that, *HLA* executes a push step on u (Line 8, Lines 19-24). Once the propagation terminates, it updates the reserve $\pi(u)$ for all $u \in D$ through edge-focused weighted degree normalization (Lines 9-10). Finally, similar to Lines 10-11 of Algorithm 1, Algorithm 2 also performs the sweep-cut operation to obtain the target cluster C (Lines 11-12). Note that Line 11 only sorts all non-zero entries in π such that $\pi_1 \geq \pi_2 \geq \dots \geq \pi_{\sup(\pi)}$ ($\sup(\pi)$ is the number of the non-zero entries in π and $\sup(\pi) \ll n$ [2]), which guarantees the locality of Algorithm 2.

THEOREM 4.4. *Given a temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal motif M_δ , and two parameters α and ϵ , the time complexity and space complexity of Algorithm 2 are $O(\frac{\Delta_{\max}}{\alpha\epsilon})$ and $O(\frac{1}{\alpha\epsilon})$, respectively. Where $\Delta(u)$ is the time cost of calculating the edge-focused weighted degree of node u and $\Delta_{\max} = \max\{\Delta(u) | u \in V\}$.*

PROOF. The time cost of Algorithm 2 is mainly consumed in calculating the edge-focused weighted degree of nodes (Line 7) and performing push operations on nodes (Line 8). Let u_i be the node to

Algorithm 2 Heuristic Local Algorithm (HLA)

Input: A temporal graph $\mathcal{G}(V, \mathcal{E})$, a δ -temporal motif M_δ , a source node s and two parameters α and ϵ
Output: A cluster C

```

1:  $r(s) \leftarrow 1$ ;  $r(u) \leftarrow 0$  for all  $u \neq s$ ;  $\pi(u) \leftarrow 0$  for all  $u \in V$ 
2:  $Q \leftarrow \{s\}$ ;  $D \leftarrow \emptyset$ ;  $G \leftarrow$  the de-temporal graph of  $\mathcal{G}$ ;  $\bar{W} \leftarrow$  an empty directed sparse matrix
3: while  $Q \neq \emptyset$  do
4:    $u \leftarrow Q.pop()$ 
5:   if  $u \notin D$  then
6:      $D \leftarrow D \cup \{u\}$ 
7:      $EFDEGREE(u)$ 
8:      $PUSH(u)$ 
9:   for  $u \in D$  and  $EFD(u) \neq 0$  do
10:     $\pi(u) \leftarrow \pi(u)/EFD(u)$ 
11:    $\pi_i \leftarrow$  the vertex with the  $i$ -th largest non-zero value in  $\pi$ 
12:    $C \leftarrow \arg \min \psi_{\bar{W}}(C_i)$ , where  $C_i = \{\pi_1, \pi_2, \dots, \pi_i\}$  and  $\psi_{\bar{W}}(C_i)$  is the edge-based conductance of  $C_i$  in  $\bar{W}$ 
13:   return  $C$ 
14: function  $EFDEGREE(u)$ 
15:    $EFD(u) \leftarrow 0$ 
16:   for each edge  $(u, v) \in G$  do
17:      $(\bar{W})_{uv} \leftarrow$  the edge-focused weight  $(W_{ef})_{uv}$  by Equation 1
18:      $EFD(u) \leftarrow EFD(u) + (\bar{W})_{uv}$ 
19: function  $PUSH(u)$ 
20:   if  $EFD(u) > 0$  and  $r(u) \geq \epsilon \cdot EFD(u)$  then
21:     for each  $v \in \{w | (\bar{W})_{uw} \neq 0\}$  do
22:        $r(v) \leftarrow r(v) + (1 - \alpha)r(u) \cdot (\bar{W})_{uv}/EFD(u)$ 
23:        $Q.push(v)$ 
24:    $\pi(u) \leftarrow \pi(u) + \alpha r(u)$ ,  $r(u) \leftarrow 0$ 

```

be pushed in the i th push operation and n_i be the number of nodes that receive propagation during the i -th push operation is performed. When the i th push operation is performed (i.e., $r(u) \geq \epsilon \cdot EFD(u)$ in Line 20), the residue $r(u_i) \geq \epsilon n_i$ due to $EFD(u_i) \geq n_i$, so $\|r\|_1$ decreased by at least $\alpha \epsilon n_i$ (Lines 21-24). Since $\|r\|_1 = 1$ initially (Line 1) and $\|r\|_1$ is at most 1 throughout the entire push process because r always decreases (Lines 22 and 24), we have $\alpha \epsilon \sum_{i=1} n_i \leq 1$ and $\sum_{i=1} n_i \leq \frac{1}{\alpha \epsilon}$. Therefore, the time complexity of all push operations in Algorithm 2 is $O(\frac{1}{\alpha \epsilon})$. On the other hand, we only calculate the edge-focused weighted degree $EFD(u)$ for vertex u in the queue Q instead of all nodes in advance. Thus, by Line 23, a total of $O(\frac{1}{\alpha \epsilon})$ vertices have ever been in Q . As a result, the time complexity of all calculations of the edge-focused weighted degree in Algorithm 2 is $O(\frac{\Delta_{\max}}{\alpha \epsilon})$. The sorting and sweep-cut costs over the nonzero entries of π are local operations and are dominated by the on-demand edge-focused weight computations. So, the overall time complexity of Algorithm 2 is $O(\frac{\Delta_{\max}}{\alpha \epsilon})$. Regarding space complexity, through the above proof process, we find that Algorithm 2 visits at most $O(\frac{1}{\alpha \epsilon})$ nodes, so it requires $O(\frac{1}{\alpha \epsilon})$ space to maintain r , π , Q , and D . Hence, the space complexity of Algorithm 2 is $O(\frac{1}{\alpha \epsilon})$. $\square$

Remark. While the proposed *HLA* lacks a formal guarantee of accuracy (though its practical performance is comparable to the baselines in our empirical results) because it is based on the heuristic local clustering framework [2, 42, 76], its main advantage is being a strictly pure local, index-free, and on-demand algorithm. It is worth noting that although the theoretical worst-case time complexity bound in Theorem 4.4 involves a global parameter $\Delta_{\max}$, this is strictly for analytical purposes to upper-bound the cost. In practice, *HLA* never precomputes or scans the entire graph to obtain $\Delta_{\max}$. Besides, unlike existing local higher-order methods that rely on a globally precomputed motif index or weighted graph, *HLA* answers a query without any global preprocessing. As a result, the actual runtime of *HLA* depends exclusively on the local degrees of the

Table 3: Dataset statistics.

Dataset	$n = V $	$m = \mathcal{E} $	$\bar{m} = E $	Time Span
<i>CM</i>	1,893	59,794	20,292	193 days
<i>EE</i>	986	327,336	24,929	903 days
<i>MO</i>	24,668	390,348	227,933	2,350 days
<i>SA</i>	30,278	445,390	56,115	338 days
<i>UB</i>	152,599	723,628	541,751	2,613 days
<i>FW</i>	43,953	851,913	262,631	1,561 days
<i>SU</i>	189,191	1,106,530	852,335	2,773 days
<i>SO</i>	2,572,345	47,895,496	34,868,599	2,774 days

actively explored nodes during the diffusion process, making it especially suitable for massive real-world temporal graphs.

5 EXPERIMENTAL EVALUATION

5.1 Experimental Setup

Datasets. We evaluate our solutions on eight publicly-available datasets (Table 3), which are widely used benchmarks for temporal motif analysis and temporal graph clustering [19, 38, 48, 58–60, 63, 68, 72, 75]. Specifically, *CM* (CollegeMsg) contains message exchanges on a Facebook-like platform at UC Irvine. *EE* (email-Eu-core-temporal) consists of email interactions among users within a European research institution. *SA* (SMS-A) contains the short message interaction among users. *FW* (facebook-wall) is the wall posts between users on the social network Facebook located in the New Orleans region. *MO* (sx-mathoverflow), *UB* (sx-askubuntu), *SU* (sx-superuser), and *SO* (sx-stackoverflow) are temporal interaction networks between users on Math Overflow, Ask Ubuntu, Super User, and Stack Overflow, in terms of comments, questions, and answers. All datasets can be downloaded from <https://networkrepository.com/> and <https://snap.stanford.edu/data/>. Note that the self-loops are deleted, and the largest weakly connected components of these datasets are used in the experiments.

Competitors. *AGA* (Algorithm 1) and *HLA* (Algorithm 2) are our proposed methods. Besides, the following ten well-known competitors are implemented for comparison. (1) Lower-order temporal graph clustering: *TSC* [65], *TMO* [54], and *TDS* [57]. *TSC* is rooted in spectral clustering, whereas *TMO* and *TDS* are based on modularity optimization and density optimization on temporal networks, respectively; (2) Higher-order static graph clustering: *HSC* [6, 30, 70], *HMO* [4, 29], and *HDS* [13, 26, 67]. *HSC*, *HMO* and *HDS* apply higher-order spectral clustering, modularity optimization and density optimization to the de-temporal graph, respectively; (3) Higher-order temporal graph clustering: *THDS* [63], *THSC* [6], *THLC* [76], and *THPR* [41]. *THDS* optimizes temporal motif density, while the remaining three optimize temporal motif conductance (Definition 6). We employ the SOTA exact temporal motif mining algorithm² proposed by Mackey et al. [51] for counting δ -instances, which is more general and efficient compared to other methods [68, 78]. The core innovation of our paper lies not in making temporal motif counting faster, but in a deliberate redesign that replaces the existing enumeration-based approach with our proposed novel counting-based clustering framework. This design is intentionally forward-compatible, allowing for the seamless adoption of more efficient counting techniques as they emerge.

Parameters. The six temporal motifs (illustrated in Figure 4) we utilize are broadly representative across most applications

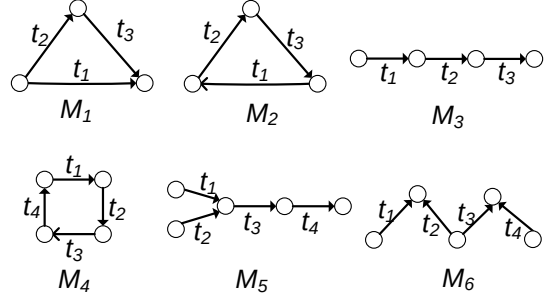


Figure 4: Temporal motifs used for experimental evaluation. $t_1 \leq t_2 \leq t_3 \leq t_4$. The temporal duration δ is treated as a parameter.

[19, 38, 48, 58, 59, 63, 68, 72, 75]. For example, M_3 represents a temporal path involving three consecutive interactions at four different nodes. This motif is fundamental for understanding the flow of information or influence propagation in temporal networks, such as the spread of rumors in social media or transaction chains in financial systems. Clusters enriched with such motifs can reveal chains of influence or potential spreading cascades. M_5 represents a more complex and branched temporal chain structure. In particular, M_5 can represent scenarios in which two different individuals contact a key person who then relays or distributes information to others. Thus, in organizational or collaboration networks, M_5 may reflect task assignment or escalation patterns. In fraud or cybersecurity, M_5 can reveal coordinated attempts converging on a target, followed by subsequent suspicious activity. By including M_5 , our approach can cluster these branching and multi-source temporal flows. These motifs encompass various topologies, including triangles, lines, cycles, and more intricate patterns, with the largest motifs presenting particularly challenging inputs. Following [38, 51, 58, 68], we set M_2 as the query temporal motif and δ to one hour by default. Due to constraints related to extended running times and limited space, our evaluation is confined to these specific motifs. Nevertheless, it is important to note that our algorithms are versatile and can be readily applied to any temporal motif without modification. Unless specified otherwise, following [2, 46, 73], we also set $\alpha = 0.2$ and $\epsilon = \frac{1}{m}$. Given that both *THLC* and *HLA* require a source vertex as input, we ensure robustness by randomly selecting 50 vertices to serve as source vertices and then report the average runtime and clustering qualities. For all other baselines, we use their default parameters.

5.2 Clustering Efficiency Evaluation

Our efficiency evaluation focuses on algorithms that directly optimize temporal motif conductance: *THSC*, *THLC*, *THPR*, *AGA*, and *HLA*. They share our objective and all require temporal motif computation. Other baselines (*TSC*, *TMO*, *TDS*, *HSC*, *HMO*, *HDS*, and *THDS*) optimize different objectives, so their runtimes are not directly comparable here. Nevertheless, we include all of them in the next section’s quality evaluation for a broader empirical comparison.

Exp-1: Running time of different temporal motif conductance algorithms. Figure 5 shows the running time of *THSC*, *THLC*, *THPR*, *AGA*, and *HLA* with varying temporal motifs, with analogous trends observed across different δ (details in [3]). As can be seen, we have: (1) *HLA* is consistently faster than other methods because it is a pure local algorithm. Unlike alternatives, it does not precompute the weight $(W_{ef})_{uv}$ for every edge (u, v) ; instead, it calculates $(W_{ef})_{uv}$ only upon accessing the edge (u, v) . Consequently, *HLA*’s runtime mainly depends on the locally explored region rather than the total graph size (Table 1). (2) *AGA* is the runner-up on all datasets and

²Although approximation algorithms [48, 61, 62, 72] can improve computational efficiency, analyzing their theoretical guarantees for clustering quality remains challenging and will be considered as future work.

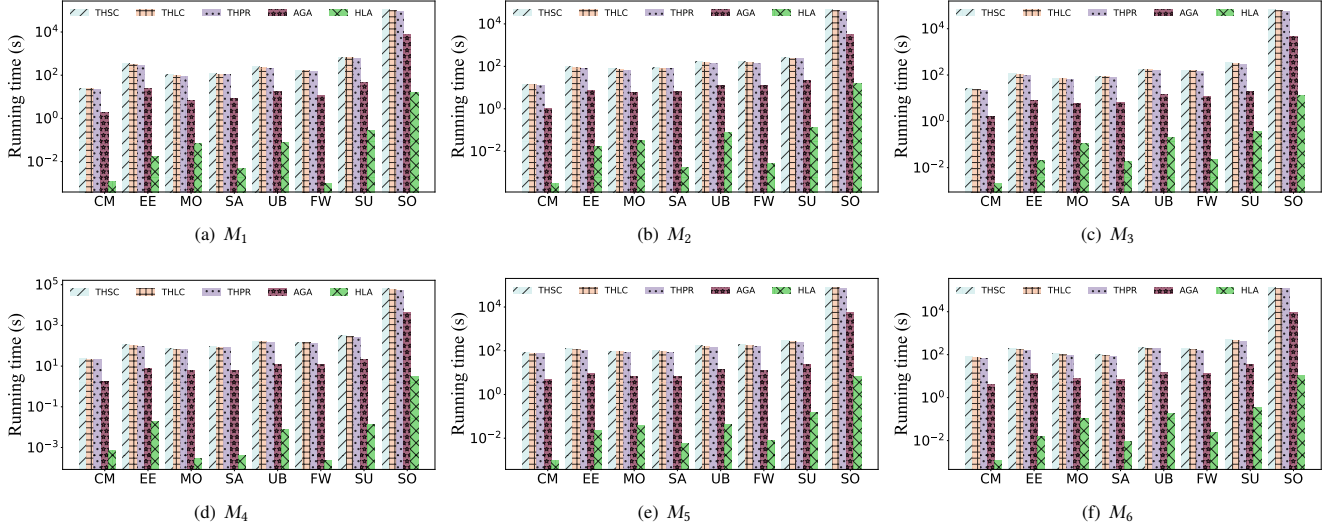


Figure 5: Running time of different temporal motif conductance algorithms with varying temporal motif.

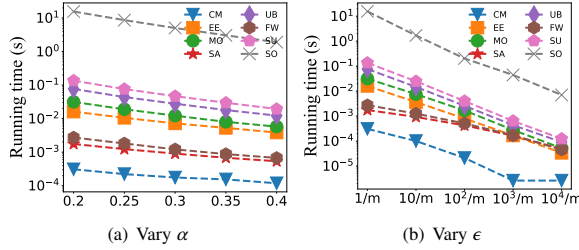


Figure 6: Running time of *HLA* with varying parameters.

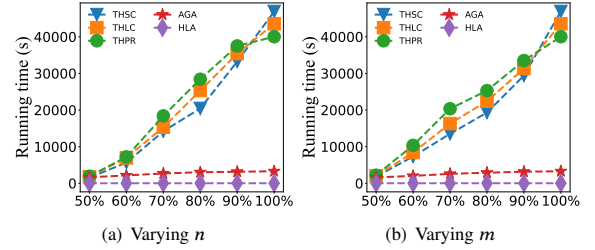


Figure 7: Scalability testing.

all temporal motifs. The efficiency of *AGA* can be attributed to its novel edge-focused weighted graph projection method, which can be efficiently implemented using the powerful temporal motif counting paradigm. However, *THSC*, *THLC*, and *THPR* depend on the exhaustive enumeration of all temporal motif instances, making them significantly slower (Table 1). In particular, our algorithms (i.e., *AGA* and *HLA*) achieve the speedups of 1 ~ 4 orders of magnitude over these enumeration-based methods in most cases. For example, on *SO* and *M6*, *AGA* and *HLA* take 9154 and 10 seconds to obtain the result, while *THSC*, *THLC*, and *THPR* require 139510, 126581, 121248 seconds, respectively. These results give preliminary evidence that the proposed solutions are highly efficient in practice.

Exp-2: Running time of *HLA* with varying parameters. Figure 6 illustrates how the parameters α and ϵ affect the running time of *HLA* with *M2* as the query temporal motif and $\delta = 1$ hour. Other query temporal motifs and δ can also obtain similar results. As can be seen, we have (1) the running time of *HLA* decreases with increasing α . An intuitive explanation is that when α increases, each vertex u that needs to transmit information will pass on a smaller residual amount to its neighbors (Line 22 of Algorithm 2), resulting in a lower probability for its neighbors to be executed in the next round of information transmission (Line 20 of Algorithm 2). Thus, when α increases, fewer nodes are visited, resulting in the locality of *HLA* being stronger (details in Algorithm 2). (2) The running time of *HLA* decreases with increasing ϵ . An intuitive explanation is that when ϵ increases, each vertex u has a smaller probability of being able to transmit information (Line 20 of Algorithm 2), resulting in fewer

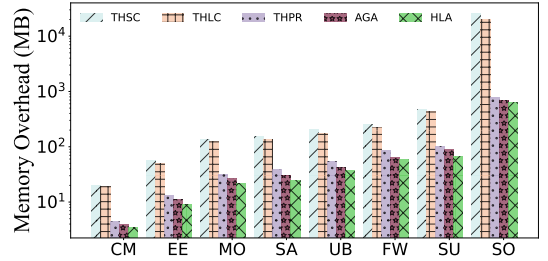


Figure 8: Memory overhead (excluding the graph itself).

neighbors being visited and stronger locality (details in Algorithm 2). Thus, these results align with our theoretical analysis (Table 1).

Exp-3: Scalability testing. We further investigated the scalability of our approaches using eight synthetically generated graphs. Figure 7 shows the runtime for query motif *M2* at $\delta = 1$ hour (other settings exhibited similar patterns). These graphs are produced by sampling edges or nodes from the *SO* dataset, which has the highest edge volume, at percentages from 50% to 100%. The results confirm that our algorithms, *AGA* and *HLA*, scale efficiently with increasing graph size. In contrast, the baselines show unstable performance, with runtimes that fluctuate significantly. This is attributable to their non-linear or exponential time complexity concerning n and m (Table 1), showing that our methods are uniquely suited for massive graphs.

Table 4: Clustering quality of various clustering methods. The best and second-best results are marked in **bold** and underlined, respectively. Smaller *TMC* and *TC* indicate higher clustering quality, while larger *TD* indicates higher clustering quality. OOT indicates the method cannot obtain the result within two days.

TMC/TC/TD	CM	EE	MO	SA	UB	FW	SU	SO
TSC	0.6/ 0.20 /2.66	0.45/ 0.20 /279.82	0.87/ 0.17 /2.80	0.81/0.41/24.07	0.69/ 0.03 /2.54	0.58/ 0.02 /227.0	0.97/ 0.02 /6.85	0.83/ 0.07 /2.00
TMO	0.56/0.85/28.71	0.29/0.42/656.38	0.75/0.31/6.99	0.64/ 0.01 /59.63	0.71/0.28/3.16	0.52/0.19/29.03	0.70/0.23/2.52	0.61/0.23/2.95
TDS	0.67/0.83/ 243.61	0.54/0.43/ 4878.0	0.70/0.51/ 547.89	0.69/ 0.01 / 6063.0	0.71/0.55/ 119.09	0.52/0.24/197.68	0.69/0.53/ 153.47	0.67/0.49/ 535.03
HSC	0.71/0.89/9.03	0.79/0.66/10.46	0.82/0.89/52.68	0.68/0.69/23.58	0.95/0.81/64.79	0.96/0.47/163	0.91/0.46/46.82	OOT
HMO	0.74/0.88/2.34	0.75/0.72/112.30	0.72/0.88/56.11	0.77/0.59/21.06	0.94/0.73/114.06	0.72/0.52/35.11	0.79/0.76/30.04	OOT
HDS	0.76/0.88/30.21	0.72/0.67/45.18	0.59/0.86/117.35	0.87/0.71/47.08	0.95/0.66/67.08	0.95/0.67/114.15	0.92/0.73/36.56	OOT
THDS	0.10/0.85/44.22	0.13/0.40/801.33	0.22/0.90/229.6	0.68/0.47/203.0	0.24/0.92/45.15	0.50/0.39/ 890.66	0.27/0.92/32.71	0.30/0.99/16.33
THSC	0.01 /0.52/37.46	0.08/0.39/802.0	0.09/0.98/17.5	0.95/0.47/213.0	0.09/0.92/44.61	0.66/0.79/17.33	0.12/0.90/61.72	0.15/0.96/9.28
THLC	0.01 /0.85/38.28	0.05/0.36/139.77	0.12/0.87/150.33	0.95/0.46/190.0	0.06/0.92/47.01	0.64/0.515/83.2	0.17/0.95/22.78	0.32/0.80/115.46
THPR	0.25/0.83/28.5	0.06/0.38/304.06	0.12/0.95/51.48	0.95/0.91/20.0	0.06/0.91/42.86	0.66/0.79/17.33	0.05/0.972/11.41	0.22/0.85/16.66
AGA	0.01 /0.52/47.46	0.03 /0.34/805.0	0.03 /0.48/270.5	0.50 /0.37/223.0	0.03 /0.12/44.61	0.33 /0.09/227.33	0.05/0.20/63.72	0.07 /0.16/159.28
HLA	0.01 /0.86/20.70	<u>0.04</u> /0.39/250.12	0.12/0.75/260.31	<u>0.60</u> /0.42/162.85	<u>0.04</u> /0.85/92.47	<u>0.40</u> /0.15/199.31	0.03 /0.8/49.54	<u>0.12</u> /0.51/259.83

Exp-4: Memory overhead comparison. The memory consumption of all algorithms, evaluated with query motif M_2 and $\delta = 1$ hour, is compared in Figure 8; results for other parameter settings follow a consistent pattern. As theoretically predicted, our proposed algorithms, *AGA* and *HLA*, significantly reduce memory usage across all datasets, outperforming all baselines by up to an order of magnitude. A case in point is the *SO* dataset, where *AGA* and *HLA* use only 698 MB and 643 MB, respectively, in stark contrast to the substantial 25628 MB and 20226 MB required by *THSC* and *THLC*. This efficiency originates from our edge-focused weighted graph projection, which confines memory complexity to $O(m)$ by assigning weights solely to the original edges. Conversely, *THSC* and *THLC* incur a much higher cost of $O(\min n^2, \binom{k}{2} \text{num}(M_\delta))$ due to their construction of hyperedges for motif instances. Furthermore, *HLA* proves slightly more efficient than *AGA* and *THPR*. This is because *THPR* maintains some auxiliary structures, such as temporal motif resident information for each vertex (Definition 8), and *AGA* requires additional space to store the intermediate variables $(W_{ef})_{(u,v)}$ for all edge (u,v) ; instead, *HLA* only store $(W_{ef})_{uv}$ upon accessing the edge (u,v) . The results provide strong experimental validation for our theoretical memory complexity claims in Table 1.

5.3 Clustering Quality Evaluation

Evaluating the quality of temporal clustering poses greater challenges than its static counterpart, primarily due to the absence of ground-truth clusters for temporal networks [46, 60, 65]. Fortunately, we can adopt the following two widely used temporal structure metrics [44–46, 60, 65, 82] for qualitative testing: temporal density (*TD*) and temporal conductance (*TC*). Specifically, let C be the identified cluster, the two metrics are defined as follows: $TD(C) = |\{(u,v,t) \in \mathcal{E} | u,v \in C\}|/|C|$ and $TC(C) = |\{(u,v,t) \in \mathcal{E} | u \in C, v \in \bar{C}\}|/\sum_{u \in C} |\{(u,v,t) \in \mathcal{E}\}|$. Clearly, *TD* computes the temporal density of the internal structure of the cluster, *TC* measures the temporal separability of the cluster. Thus, the larger *TD*(C) (resp., smaller *TC*(C)) indicates the denser inside (resp., sparser outside) of C . Besides, we also evaluate the clustering quality using the proposed temporal motif conductance (*TMC*), as defined in Definition 6. As a consequence, smaller *TMC* and *TC* indicate higher clustering quality, while larger *TD* indicates higher clustering quality. It is worth noting that these metrics capture complementary aspects of temporal clustering quality. *TMC* evaluates higher-order temporal motif preservation, which is the objective studied in this paper. *TC* and *TD*, on the other hand, are lower-order temporal edge-level metrics that respectively measure external temporal separability and internal temporal density. Our algorithms do not directly optimize *TC* or *TD*. Therefore, their performance on *TC* and *TD* serves as an additional structural validation rather than a direct consequence of optimizing the proposed objective.

Exp-5: Clustering quality of various clustering methods. Since traditional lower-order temporal (or higher-order static) graph clustering methods (i.e., *TSC*, *TMO*, *TDS*, *HSC*, *HMO*, and *HDS*) do not rely on temporal motifs, Table 4 only shows results for M_2 as the query temporal motif and $\delta = 1$ hour. The empirical results of other settings are placed in the [3] due to space limitations. For the *TMC* metric, we have (1) *AGA* outperforms all other methods on seven out of eight datasets. This superiority is attributed to *AGA*’s ability to identify clusters with quadratic Cheeger-type guarantee, whereas other methods provide poor or no theoretical guarantees (Table 1). (2) *HLA* is superior to all the other methods except *AGA*. This is because *HLA* builds upon the edge-focused counting principle of *AGA*; it maintains strong practical performance despite lacking a theoretical guarantee. (3) *TSC*, *TMO*, *TDS*, *HSC*, *HMO*, and *HDS* exhibit poor clustering quality. This is because they ignore the crucial temporal information or higher-order information. For the *TC&TD* metric, we have: (1) the proposed algorithm *AGA* achieves the best overall performance without the situation where one metric is excellent but another is poor. (2) *TSC* has good *TC* value but has poor *TD* value. On the contrary, *TDS* has good *TD* value but poor *TC* value. This is because *TSC* merely considers the external temporal separateness but ignores the internal temporal density, *TDS* merely considers the internal temporal density but ignores the external temporal separateness. (3) Other methods have the worst performance. This is because either they are static methods, which fail to take into account temporal information, or they do not consider the temporal dense inside or the temporal sparse outside. These results show that existing models cannot optimize our proposed objective function well, and our algorithm is much denser and more separable in temporal feature than existing models.

Remark. Optimizing *TMC*, *TD*, and *TC* simultaneously is very challenging. The reasons are as follows. *TD* and *TC* are the objective functions optimized by *TDS* and *TSC*, respectively, so it is reasonable that they achieve the best performance on the corresponding metric. Similarly, *TMC* is the metric we proposed, thus our superior performance on it is also expected. However, as shown in Table 4, *TD* and *TC* are inherently in a trade-off relationship, that is, no single method can achieve the optimal performance on both simultaneously. Our method can find the best overall performance balance between *TD* and *TC*, avoiding the extreme situations where existing baseline models have to sacrifice one for the other, aligning with similar observations reported in [46]. As a consequence, our model is a trade-off among them.

Exp-6: Quality comparison between *AGA* and *HLA*. Figure 9 compares the clustering quality of the approximate global algorithm (*AGA*) against the heuristic local algorithm (*HLA*). Using the temporal motif conductance ρ_{aga} from *AGA* as a benchmark, we

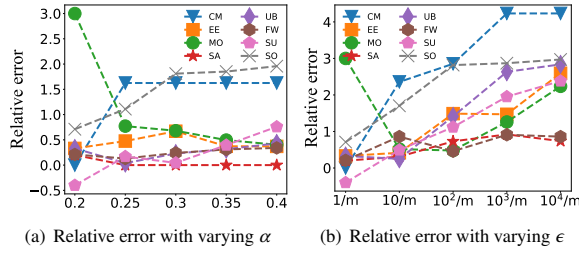


Figure 9: Quality comparison between AGA and HLA.

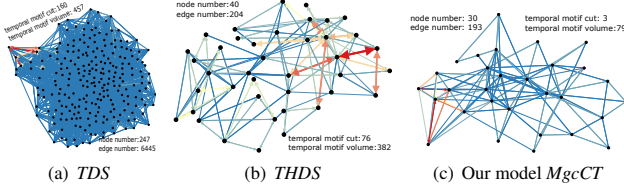


Figure 10: Case studies. The thicker and darker the edge, the more temporal cycles it participates in.

define the relative error (RE) of *HLA* as $RE = (\rho_{hla} - \rho_{aga}) / \rho_{aga}$. As can be seen, we have (1) the RE is consistently bounded between -0.5 and 4, indicating the robust practical performance of *HLA*. (2) The RE generally increases with parameters α and ϵ . This trend is attributed to the role of these parameters in limiting the exploration scope of *HLA*; higher values restrict the search to a narrower neighborhood, increasing the risk of converging to a suboptimal local solution. Thus, the heuristic local algorithm *HLA* can identify high-quality clusters in practice.

Exp-7: Case studies. We conduct case studies on the Bitcoin dataset [35], which comprises 3,775 nodes and 24,180 temporal edges. Here, nodes represent users and temporal edges indicate when the corresponding users conducted the transaction through the well-known Bitcoin Alpha platform. To demonstrate the utility of our model, we select the temporal cycle M_4 in Figure 4 as the query motif, as it is a recognized indicator for identifying money laundering activities [34, 37, 66]. Figure 10 visualizes the clusters identified by each method. Note that clusters detected by *TMO*, *TSC*, *HMO*, *HSC*, and *HDS* are either excessively large or too small for effective visualization. For instance, clusters identified by *TMO* and *HMO* contain over 1,000 vertices, reflecting the modularity resolution limitation [15]. Furthermore, Table 4 confirms their poor performance in this context. In this case, *TDS* identifies a cluster of 247 users and 6,445 edges, densely connected via dyadic temporal edges but lacking higher-order temporal cycles (Figure 10(a)). Consequently, the number of temporal cycle instances is an order of magnitude lower than the number of edges. A detailed inspection of Figure 10(a) reveals that *TDS* mainly captures a temporally dense dyadic-edge region, but it does not concentrate temporal-cycle structures well. Although *THDS* identifies more temporal-cycle instances inside the cluster (Figure 10(b)), the resulting cluster also has many temporal-cycle connections to outside vertices, violating the external sparsity requirement and leading to a less separated result [25, 31, 36, 40, 50]. In contrast, the cluster identified by our model *MgcCT* in Figure 10(c) achieves a better balance between internal temporal-cycle preservation and external temporal-cycle separability. Specifically, it preserves suspicious temporal-cycle structures inside the cluster while keeping the number of cross-cluster temporal cycles small. This suggests that *MgcCT* can reveal

compact temporal-cycle patterns that are structurally consistent with suspicious circulation behaviors reported in financial temporal networks [34, 47, 75].

6 RELATED WORK

Static Graph Clustering. Graph clustering on static networks has received much attention over past decades [16, 36, 52]. Representative models include edge cut based (e.g., ratio cut or normalized cut [71]), null model based (e.g., modularity [55]), and subgraph cohesiveness based (e.g., clique or k -core [10]). The above methods mainly focus on the internal or external dyadic edges of the cluster, resulting in that cannot capture the more informative and semantic higher-order structure. Until recently, motifs (i.e., small subgraphs) have been shown to be indispensable for modeling and understanding the higher-order organization of complex networks [6]. Thus, a wide spectrum of motif-based higher-order graph clustering methods have been proposed [4–6, 13, 29, 30, 67, 70, 74, 76, 81]. For example, motif modularity was proposed to extend the traditional modularity by optimizing the difference between the fraction of motif instances within the cluster and the fraction in a random network preserving the same degree of vertices [4, 29]. Higher-order densest subgraph model was proposed where the density is defined as the number of motif instances divided by the size of vertices [13, 26, 67]. However, all these methods cannot be directly applied to our problem due to they ignore the important time information.

Temporal Graph Clustering. Temporal dynamics constitute a critical dimension for modeling real-world systems, offering profound insights into graph clustering [11, 54, 57, 65]. For example, Silva et al. [65] generalized sparsest and normalized cuts to temporal graphs by introducing temporal smoothness constraints on cut evolution. Mucha et al. [54] developed temporal modularity to quantify clustering quality across time slices, rewarding stable partitions with high instantaneous modularity and penalizing vertex migrations between clusters. Qin et al. [60] investigated bursting clusters in temporal networks, where clusters exhibit transient density peaks. Yu et al. [77] proposed an index-based method with pruning strategies to accelerate historical k -core discovery. However, these approaches are fundamentally limited by their focus on low-order connectivity patterns, rendering them inapplicable to our problem domain. Recently, Fu et al. [17, 18] studied the higher-order graph clustering on dynamic graphs, which aims to maintain the new higher-order clusters when vertices/edges are inserted/deleted. Thus, their objective function considers *only one timestamp*, while our model considers higher-order clusters that allow *span a time interval*. As a result, these dynamic higher-order graph clustering methods are orthogonal to ours [20, 46, 57, 60, 63].

7 CONCLUSION

To capture the interplay of continuous topological-temporal dependencies and higher-order structural patterns in temporal clusters, we introduce a novel metric, temporal motif conductance, quantifying cluster cohesion through motif-constrained random walk dynamics. We formalize our objective as minimizing this conductance and rigorously establish its NP-hardness. To address this computational challenge, we propose two complementary algorithms: a global approximation algorithm *AGA* achieving a quadratic Cheeger bound, and a scalable heuristic local algorithm *HLA* that bypasses full graph traversal, thereby achieving a superior balance between runtime efficiency and output quality. Extensive experiments on eight real-world temporal networks demonstrate that our framework outperforms ten competitors in both clustering efficiency and quality.

REFERENCES

- [1] Noga Alon and Vitali D Milman. 1985. λ_1 , isoperimetric inequalities for graphs, and superconcentrators. *Journal of Combinatorial Theory, Series B* 38, 1 (1985), 73–88.
- [2] Reid Andersen, Fan R. K. Chung, and Kevin J. Lang. 2006. Local Graph Partitioning using PageRank Vectors. In *FOCS*. 475–486.
- [3] Anonymous. 2026. full version available at: <https://github.com/longlonglin/MgcCT>. (2026).
- [4] Alex Arenas, Alberto Fernandez, Santo Fortunato, and Sergio Gomez. 2008. Motif-based communities in complex networks. *Journal of Physics A: Mathematical and Theoretical* 41, 22 (2008), 224001.
- [5] Austin R. Benson, David F. Gleich, and Jure Leskovec. 2015. Tensor Spectral Clustering for Partitioning Higher-order Network Structures. In *SDM*. 118–126.
- [6] Austin R. Benson, David F. Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.
- [7] Filippo Maria Bianchi, Daniele Grattarola, and Cesare Alippi. 2020. Spectral Clustering with Graph Neural Networks for Graph Pooling. In *ICML*. 874–883.
- [8] Xin-Wei Cai, Xiangyu Ke, Kai Wang, Lu Chen, Tianming Zhang, Qing Liu, and Yunjun Gao. 2023. Efficient Temporal Butterfly Counting and Enumeration on Temporal Bipartite Graphs. *Proc. VLDB Endow.* 17, 4 (2023), 657–670.
- [9] Abdulkadir Celikkanat, Nikolaos Nakis, and Morten Mørup. 2024. Continuous-Time Graph Representation with Sequential Survival Process. In *AAAI*. 11177–11185.
- [10] Lijun Chang and Lu Qin. 2019. Cohesive Subgraph Computation Over Large Sparse Graphs. In *Proceedings of ICDE*. 2068–2071.
- [11] Yun Chi, Xiaodan Song, Dengyong Zhou, Koji Hino, and Belle L. Tseng. 2007. Evolutionary spectral clustering by incorporating temporal smoothness. In *KDD*. 153–162.
- [12] Karen D. Devine, Erik G. Boman, and George Karypis. 2006. Partitioning and Load Balancing for Emerging Parallel Applications and Architectures. In *Parallel Processing for Scientific Computing*. 99–126.
- [13] Yixiang Fang, Kaiqiang Yu, Reynold Cheng, Laks V. S. Lakshmanan, and Xuemin Lin. 2019. Efficient Algorithms for Densest Subgraph Discovery. *Proc. VLDB Endow.* 12, 11 (2019), 1719–1732.
- [14] Miroslav Fiedler. 1973. Algebraic connectivity of graphs. *Czechoslovak mathematical journal* 23, 2 (1973), 298–305.
- [15] Santo Fortunato and Marc Barthélemy. 2007. Resolution limit in community detection. *Proceedings of the national academy of sciences* 104, 1 (2007), 36–41.
- [16] Santo Fortunato and Darko Hric. 2016. Community detection in networks: A user guide. *CoRR abs/1608.00163* (2016).
- [17] Dongqi Fu, Dawei Zhou, and Jingrui He. 2020. Local Motif Clustering on Time-Evolving Graphs. In *SIGKDD*. 390–400.
- [18] Dongqi Fu, Dawei Zhou, Ross Maciejewski, Arie Croitoru, Marcus Boyd, and Jingrui He. 2023. Fairness-Aware Clique-Preserving Spectral Clustering of Temporal Graphs. In *WWW*. 3755–3765.
- [19] Zhongqiang Gao, Chuanqi Cheng, Yanwei Yu, Lei Cao, Chao Huang, and Junyu Dong. 2022. Scalable motif counting for large-scale temporal graphs. In *ICDE*. 2656–2668.
- [20] Aristides Gionis, Lutz Oettershagen, and Ilie Sarpe. 2024. Mining Temporal Networks. In *WWW*. 1260–1263.
- [21] David F. Gleich and C. Seshadhri. 2012. Vertex neighborhoods, low conductance cuts, and good seeds for local community methods. In *KDD*. 597–605.
- [22] Gene H Golub and Charles F Van Loan. 2013. *Matrix computations*. JHU press.
- [23] Saket Gurukar, Sayan Ranu, and Balaraman Ravindran. 2015. COMMIT: A Scalable Approach to Mining Communication Motifs from Dynamic Networks. In *SIGMOD*. 475–489.
- [24] Liangzhe Han, Xiaojian Ma, Leilei Sun, Bowen Du, Yanjie Fu, Weifeng Lv, and Hui Xiong. 2022. Continuous-Time and Multi-Level Graph Representation Learning for Origin-Destination Demand Prediction. In *KDD*. 516–524.
- [25] Yue He, Longlong Lin, Pingpeng Yuan, Ronghua Li, Tao Jia, and Zeli Wang. 2024. CCSS: Towards conductance-based community search with size constraints. *Expert Syst. Appl.* 250 (2024), 123915.
- [26] Yizhang He, Kai Wang, Wenjie Zhang, Xuemin Lin, and Ying Zhang. 2023. Scaling Up k-Clique Densest Subgraph Detection. *Proc. ACM Manag. Data* 1, 1 (2023), 69:1–69:26.
- [27] Petter Holme. 2015. Modern temporal network theory: A colloquium. *CoRR abs/1508.01303* (2015).
- [28] Petter Holme and Jari Saramaki. 2012. Temporal networks. *Physics Reports* 519 (2012), 97–125.
- [29] Ling Huang, Hong-Yang Chao, and Guangqiang Xie. 2020. MuMod: A Micro-Unit Connection Approach for Hybrid-Order Community Detection. In *AAAI*. 107–114.
- [30] Shixun Huang, Yuchen Li, Zhifeng Bao, and Zhao Li. 2021. Towards Efficient Motif-based Graph Partitioning: An Adaptive Sampling Approach. In *ICDE*. 528–539.
- [31] Raj Kamal and Amitabha Bagchi. 2024. A Lovász-Simonovits Theorem for Hypergraphs with Application to Local Clustering. *Proc. ACM Manag. Data* 2, 4 (2024), 190:1–190:27.
- [32] Lauri Kovanen, Márton Karsai, Kimmo Kaski, János Kertész, and Jari Saramäki. 2011. Temporal motifs in time-dependent networks. *Journal of Statistical Mechanics: Theory and Experiment* 2011, 11 (2011), P11005.
- [33] Lauri Kovanen, Kimmo Kaski, János Kertész, and Jari Saramäki. 2013. Temporal motifs reveal homophily, gender-specific patterns, and group talk in call sequences. *Proceedings of the National Academy of Sciences* 110, 45 (2013), 18070–18075.
- [34] Rohit Kumar and Toon Calders. 2018. 2SCENT: An Efficient Algorithm to Enumerate All Simple Temporal Cycles. *Proc. VLDB Endow.* 11, 11 (2018), 1441–1453.
- [35] Srijan Kumar, Bryan Hooi, Disha Makhija, Mohit Kumar, Christos Faloutsos, and VS Subrahmanian. 2018. Rev2: Fraudulent user prediction in rating platforms. In *WSDM*. 333–341.
- [36] Jure Leskovec, Kevin J. Lang, and Michael W. Mahoney. 2010. Empirical comparison of algorithms for network community detection. In *WWW*. 631–640.
- [37] Michael Levi and Peter Reuter. 2006. Money laundering. *Crime and justice* 34, 1 (2006), 289–375.
- [38] Jiantao Li, Jianpeng Qi, Yueling Huang, Lei Cao, Yanwei Yu, and Junyu Dong. 2024. MoTTo: Scalable Motif Counting with Time-aware Topology Constraint for Large-scale Temporal Graphs. In *CIKM*. 1195–1204.
- [39] Zhiyuan Li, Xun Jian, Yue Wang, and Lei Chen. 2022. CC-GNN: A Community and Contraction-based Graph Neural Network. In *ICDM*, Xingquan Zhu, Sanjay Ranka, My T. Thai, Takashi Washio, and Xindong Wu (Eds.). 231–240.
- [40] Longlong Lin, Yue He, Wei Chen, Pingpeng Yuan, Rong-Hua Li, and Tao Jia. 2025. Effective and Efficient Conductance-Based Community Search at Billion Scale. *IEEE Trans. Big Data* 11, 6 (2025), 3170–3184.
- [41] Longlong Lin, Tao Jia, Zeli Wang, Jin Zhao, and Rong-Hua Li. 2024. PSMC: Provable and Scalable Algorithms for Motif Conductance Based Graph Clustering. In *KDD*. 1793–1803.
- [42] Longlong Lin, Ronghua Li, and Tao Jia. 2023. Scalable and Effective Conductance-Based Graph Clustering. In *AAAI*. 4471–4478.
- [43] Longlong Lin, Zeli Wang, Rong-Hua Li, Qiuyu Liu, Hongchao Qin, and Jin Zhao. 2025. Provable Higher-Order Graph Clustering: The Power of Peeling-Based Approaches. *IEEE Trans. Knowl. Data Eng.* 37, 9 (2025), 5568–5583.
- [44] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, and Hai Jin. 2022. Mining Diversified Top- k Lasting Cohesive Subgraphs on Temporal Networks. *IEEE Trans. Big Data* 8, 6 (2022), 1537–1549.
- [45] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Jifei Wang, Ling Liu, and Hai Jin. 2022. Mining Stable Quasi-Cliques on Temporal Networks. *IEEE Trans. Syst. Man Cybern. Syst.* 52, 6 (2022), 3731–3745.
- [46] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Chun-Xue Zhu, Hongchao Qin, Hai Jin, and Tao Jia. 2024. QTCs: Efficient Query-Centered Temporal Community Search. *Proc. VLDB Endow.* 17, 6 (2024), 1187–1199.
- [47] Penghang Liu, Rupam Acharyya, Robert E. Tillman, Shunya Kimura, Naoki Masuda, and Ahmet Erdem Sariyuce. 2023. Temporal Motifs for Financial Networks: A Study on Mercari, JPMC, and Venmo Platforms. *CoRR abs/2301.07791* (2023).
- [48] Paul Liu, Austin R. Benson, and Moses Charikar. 2019. Sampling Methods for Counting Temporal Motifs. In *WSDM*. 294–302.
- [49] Penghang Liu, Valerio Guarasi, and Ahmet Erdem Sariyuce. 2023. Temporal Network Motifs: Models, Limitations, Evaluation. *IEEE Trans. Knowl. Data Eng.* 35, 1 (2023), 945–957.
- [50] Yunkai Lou, Chaokun Wang, Tiankai Gu, Hao Feng, Jun Chen, and Jeffrey Xu Yu. 2021. Time-Topology Analysis. *Proc. VLDB Endow.* 14, 13 (2021), 3322–3334.
- [51] Patrick Mackey, Katherine Porterfield, Erin Fitzhenry, Sutanay Choudhury, and George Chin. 2018. A chronological edge-driven approach to temporal subgraph isomorphism. In *IEEE BigData*. 3972–3979.
- [52] Markus Maier, Ulrike von Luxburg, and Matthias Hein. 2008. Influence of graph construction on graph-based clustering measures. In *NIPS*. 1025–1032.
- [53] Marina Meila and Jianbo Shi. 2001. A Random Walks View of Spectral Segmentation. In *AISTATS*, Thomas S. Richardson and Tommi S. Jaakkola (Eds.).
- [54] Peter J Mucha, Thomas Richardson, Kevin Macon, Mason A Porter, and Jukka-Pekka Onnela. 2010. Community structure in time-dependent, multiscale, and multiplex networks. *science* 328, 5980 (2010), 876–878.
- [55] Mark EJ Newman. 2004. Fast algorithm for detecting community structure in networks. *Physical review E* 69, 6 (2004), 066133.
- [56] Giang Hoang Nguyen, John Boaz Lee, Ryan A. Rossi, Nesreen K. Ahmed, Eunye Koh, and Sunghul Kim. 2018. Continuous-Time Dynamic Network Embeddings. In *WWW*. 969–976.
- [57] Lutz Oettershagen, Athanasios L. Konstantinidis, and Giuseppe F. Italiano. 2025. An Edge-Based Decomposition Framework for Temporal Networks. In *WSDM*. 735–743.
- [58] Ashwin Paranjape, Austin R. Benson, and Jure Leskovec. 2017. Motifs in Temporal Networks. In *WSDM*. 601–610.
- [59] Noujan Pashanasangi and C Seshadhri. 2021. Faster and generalized temporal triangle counting, via degeneracy ordering. In *KDD*. 1319–1328.
- [60] Hongchao Qin, Ronghua Li, Ye Yuan, Guoren Wang, Lu Qin, and Zhiwei Zhang. 2022. Mining Bursting Core in Large Temporal Graph. *Proc. VLDB Endow.* 15, 13 (2022), 3911–3923.
- [61] Ilie Sarpe and Fabio Vandin. 2021. odeN: Simultaneous Approximation of Multiple Motif Counts in Large Temporal Networks. In *CIKM*. 1568–1577.
- [62] Ilie Sarpe and Fabio Vandin. 2021. Presto: Simple and scalable sampling techniques for the rigorous approximation of temporal motif counts. In *SDM*. 145–153.
- [63] Ilie Sarpe, Fabio Vandin, and Aristides Gionis. 2024. Scalable Temporal Motif Densest Subnetwork Discovery. In *KDD*. 2536–2547.
- [64] Jianbo Shi and Jitendra Malik. 1997. Normalized Cuts and Image Segmentation. In *CVPR*. 731–737.
- [65] Arlei Silva, Ambuj K. Singh, and Ananthram Swami. 2018. Spectral Algorithms for Temporal Graph Cuts. In *WWW*. 519–528.
- [66] Tiziano Squartini, Iman van Lelyveld, and Diego Garlaschelli. 2013. Early-warning signals of topological collapse in interbank networks. *Scientific Reports* 3 (2013).
- [67] Binta Sun, Maximilien Danisch, T.-H. Hubert Chan, and Mauro Sozio. 2020. KClust++: A Simple Algorithm for Finding k-Clique Densest Subgraphs in Large

- Graphs. *Proc. VLDB Endow.* 13, 10 (2020), 1628–1640.
- [68] Nishil Talati, Haojie Ye, Sanketh Vedula, Kuan-Yu Chen, Yuhao Chen, Daniel Liu, Yichao Yuan, David Blaauw, Alex Bronstein, Trevor Mudge, et al. 2022. Mint: An accelerator for mining temporal motifs. In *MICRO*. 1270–1287.
 - [69] David Tolliver and Gary L. Miller. 2006. Graph Partitioning by Spectral Rounding: Applications in Image Segmentation and Clustering. In *CVPR*. 1053–1060.
 - [70] Charalampos E. Tsourakakis, Jakub Pachocki, and Michael Mitzenmacher. 2017. Scalable Motif-aware Graph Clustering. In *WWW*, Rick Barrett, Rick Cummings, Eugene Agichtein, and Evgeniy Gabrilovich (Eds.). 1451–1460.
 - [71] Ulrike Von Luxburg. 2007. A tutorial on spectral clustering. *Statistics and computing* 17, 4 (2007), 395–416.
 - [72] Jingjing Wang, Yanhao Wang, Wenjun Jiang, Yuchen Li, and Kian-Lee Tan. 2020. Efficient Sampling Algorithms for Approximate Temporal Motif Counting. In *CIKM*. 1505–1514.
 - [73] Zhewei Wei, Xiaodong He, Xiaokui Xiao, Sibao Wang, Shuo Shang, and Ji-Rong Wen. 2018. TopPPR: Top-k Personalized PageRank Queries with Precision Guarantees on Large Graphs. In *SIGMOD*. 441–456.
 - [74] Tao Wu, Austin R. Benson, and David F. Gleich. 2016. General Tensor Spectral Co-clustering for Higher-Order Data. In *NIPS*, Daniel D. Lee, Masashi Sugiyama, Ulrike von Luxburg, Isabelle Guyon, and Roman Garnett (Eds.). 2559–2567.
 - [75] Yuyang Xia, Yixiang Fang, and Wensheng Luo. 2025. Efficiently Counting Triangles in Large Temporal Graphs. *Proceedings of the ACM on Management of Data* 3, 1 (2025), 1–27.
 - [76] Hao Yin, Austin R. Benson, Jure Leskovec, and David F. Gleich. 2017. Local Higher-Order Graph Clustering. In *KDD*. 555–564.
 - [77] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On Querying Historical K-Cores. *Proc. VLDB Endow.* 14, 11 (2021), 2033–2045.
 - [78] Yichao Yuan, Haojie Ye, Sanketh Vedula, Wynn Kaza, and Nishil Talati. 2023. Everest: GPU-Accelerated System For Mining Temporal Motifs. *Proc. VLDB Endow.* 17, 2 (2023), 162–174.
 - [79] Yifei Zhang, Longlong Lin, Pingpeng Yuan, and Hai Jin. 2022. Significant Engagement Community Search on Temporal Networks. In *DASFAA*. 250–258.
 - [80] Dawei Zhou, Jingrui He, Hasan Davulcu, and Ross Maciejewski. 2018. Motif-Preserving Dynamic Local Graph Cut. In *Big Data*. 1156–1161.
 - [81] Dawei Zhou, Si Zhang, Mehmet Yigit Yildirim, Scott Alcorn, Hanghang Tong, Hasan Davulcu, and Jingrui He. 2017. A Local Algorithm for Structure-Preserving Graph Cut. In *KDD*. 655–664.
 - [82] Chun-Xue Zhu, Longlong Lin, Pingpeng Yuan, and Hai Jin. 2022. Discovering Cohesive Temporal Subgraphs with Temporal Density Aware Exploration. *J. Comput. Sci. Technol.* 37, 5 (2022), 1068–1085.

A APPENDIX

A.1 Missing Proofs

Proof of Theorem 4.1. Initially, Algorithm 1 requires $O(C(\mathcal{M}_\delta))$ time to construct the edge-focused weighted matrix W_{ef} (Lines 2-4). The symmetrization step $W \leftarrow W_{ef} + W_{ef}^\top$ takes $O(\bar{m})$ time (Line 5) since W_{ef} is supported on the edges of the de-temporal graph G . Next, constructing $\mathcal{D}$ costs $O(\bar{m})$ time (Lines 6-7), and forming the Laplacian matrix $\mathcal{L}$ costs $O(\bar{m})$ time (Line 8). Subsequently, Algorithm 1 employs the well-established Lanczos iteration algorithm [22] to compute the eigenvector associated with the second smallest eigenvalue of $\mathcal{L}$, which takes $O(\bar{m})$ time (Line 9). Finally, the algorithm takes $O(n \log n)$ time to sort the indices (Line 10) and $O(\bar{m})$ time to calculate the conductance of C_i on the weighted graph W (Line 11). Consequently, the overall time complexity of Algorithm 1 is $O(C(\mathcal{M}_\delta) + \bar{m} + n \log n)$. Since the edge-focused motif counting step dominates the remaining sparse graph operations, the running time can be further reduced as $O(C(\mathcal{M}_\delta))$.

For space complexity, Algorithm 1 needs an additional $O(\bar{m})$ space to store W_{ef} , W , $\mathcal{D}$, and $\mathcal{L}$. Additionally, it requires $O(m)$ space to store the input temporal graph $\mathcal{G}$. Hence, the total space complexity of Algorithm 1 is $O(m)$ due to $m > \bar{m}$.

Proof of Theorem 4.2. For any cluster $C \subseteq V$, the edge conductance on W is $\phi_W(C) = \frac{\sum_{u \in C, v \notin C} W_{uv}}{\min\{\sum_{u \in C} \sum_{v \in V} W_{uv}, \sum_{u \in \bar{C}} \sum_{v \in V} W_{uv}\}}$ [21, 42]. By Lemma 4.1, we have $\underline{c}_\delta \text{cut}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C, v \notin C} W_{uv} \leq \bar{c}_\delta \text{cut}_{\mathcal{M}_\delta}(C)$. By Lemma 4.2, we have $\underline{d}_\delta \text{vol}_{\mathcal{M}_\delta}(C) \leq \sum_{u \in C} \sum_{v \in V} W_{uv} \leq \bar{d}_\delta \text{vol}_{\mathcal{M}_\delta}(C)$, and similarly, $\underline{d}_\delta \text{vol}_{\mathcal{M}_\delta}(\bar{C}) \leq \sum_{u \in \bar{C}} \sum_{v \in V} W_{uv} \leq \bar{d}_\delta \text{vol}_{\mathcal{M}_\delta}(\bar{C})$. Combining these inequalities gives

$$\frac{\underline{c}_\delta}{\underline{d}_\delta} \phi_{\mathcal{M}_\delta}(C) \leq \phi_W(C) \leq \frac{\bar{c}_\delta}{\bar{d}_\delta} \phi_{\mathcal{M}_\delta}(C). \quad (2)$$

Let $\phi_W^* = \min_{C \subseteq V} \phi_W(C)$. By the classical Cheeger inequality for undirected weighted graphs [1], we have

$$\frac{\lambda_2}{2} \leq \phi_W^* \leq \sqrt{2\lambda_2}. \quad (3)$$

Let $C_{\mathcal{M}_\delta}^*$ be an optimal solution for temporal motif conductance. By Eq.2, $\phi_W^* \leq \phi_W(C_{\mathcal{M}_\delta}^*) \leq \frac{\bar{c}_\delta}{\underline{d}_\delta} \phi_{\mathcal{M}_\delta}(C_{\mathcal{M}_\delta}^*) = \frac{\bar{c}_\delta}{\underline{d}_\delta} \phi_{\mathcal{M}_\delta}^*$. Together with $\lambda_2/2 \leq \phi_W^*$ from Eq.3, we obtain $\frac{\underline{d}_\delta}{2\bar{c}_\delta} \lambda_2 \leq \phi_{\mathcal{M}_\delta}^*$. Next, let C_W^* be an optimal solution for ϕ_W . By Eq.2, $\phi_{\mathcal{M}_\delta}(C_W^*) \leq \frac{\bar{d}_\delta}{\underline{c}_\delta} \phi_W(C_W^*) = \frac{\bar{d}_\delta}{\underline{c}_\delta} \phi_W^*$. Since $\phi_{\mathcal{M}_\delta}^* \leq \phi_{\mathcal{M}_\delta}(C_W^*)$ and $\phi_W^* \leq \sqrt{2\lambda_2}$ by Eq.3, we have $\phi_{\mathcal{M}_\delta}^* \leq \frac{\bar{d}_\delta}{\underline{c}_\delta} \sqrt{2\lambda_2}$. Combining the two bounds completes the proof.

Proof of Theorem 4.3. During the sweep-cut procedure in Lines 9–11 of Algorithm 1, the standard spectral clustering guarantee for undirected weighted graphs implies that the returned set C satisfies [14, 42]

$$\phi_W(C) \leq \sqrt{2\lambda_2}. \quad (4)$$

By Eq.2 in the proof of Theorem 4.2, for any cluster C , we have

$$\phi_{\mathcal{M}_\delta}(C) \leq \frac{\bar{d}_\delta}{\underline{c}_\delta} \phi_W(C). \quad (5)$$

Combining Eq.4 and Eq.5, we obtain

$$\phi_{\mathcal{M}_\delta}(C) \leq \frac{\bar{d}_\delta}{\underline{c}_\delta} \sqrt{2\lambda_2}. \quad (6)$$

By the Cheeger inequality [1], $\lambda_2/2 \leq \phi_W^*$, which implies $\lambda_2 \leq 2\phi_W^*$. Moreover, applying Eq.2 to the optimal temporal-motif-conductance cluster $C_{\mathcal{M}_\delta}^*$ gives

$$\phi_W^* \leq \phi_W(C_{\mathcal{M}_\delta}^*) \leq \frac{\bar{c}_\delta}{\underline{d}_\delta} \phi_{\mathcal{M}_\delta}^*. \quad (7)$$

Substituting Eq.7 into Eq.6, we have $\phi_{\mathcal{M}_\delta}(C) \leq \frac{\bar{d}_\delta}{\underline{c}_\delta} \sqrt{2\lambda_2} \leq \frac{\bar{d}_\delta}{\underline{c}_\delta} \sqrt{4\phi_W^*} \leq 2 \frac{\bar{d}_\delta}{\underline{c}_\delta} \sqrt{\frac{\bar{c}_\delta}{\underline{d}_\delta} \phi_{\mathcal{M}_\delta}^*}$. Since $\underline{c}_\delta$, $\bar{c}_\delta$, $\underline{d}_\delta$, and $\bar{d}_\delta$ are constants determined only by the query temporal motif, we have $\phi_{\mathcal{M}_\delta}(C) = O(\sqrt{\phi_{\mathcal{M}_\delta}^*})$ for fixed-size temporal motifs.

A.2 Temporal Motif Analysis

Temporal motifs have been widely recognized as fundamental higher-order structural units in temporal networks, with extensive research exploring their properties and applications [8, 23, 32, 34, 48, 58, 61, 62, 72]. Early foundational work by Kovanen et al. [32] introduced the first temporal motif definition that explicitly incorporates Δ -adjacency constraints between consecutive events (i.e., temporal edges), where two events are considered Δ -adjacent if their timestamp difference does not exceed Δ . Gurukar et al. [23] investigated communication motifs as recurring subgraphs with consistent information flow patterns, while Paranjape et al. [58] proposed the δ -temporal motif, which incorporates both temporal duration and ordering constraints (Definition 1). This work has inspired numerous exact algorithms for mining specific 3-edge δ -temporal motifs [19, 38, 58, 59, 75], as well as sampling-based approximation methods [48, 72]. However, these small motifs face limitations in real-world applications [49, 51, 68, 78]. For arbitrary δ -temporal motifs with flexible node and edge configurations, Mackey et al. [51] developed the SOTA BT algorithm, an exact backtracking approach based on isomorphism testing. This has subsequently motivated hardware-accelerated implementations to optimize performance [68, 78]. Many randomized algorithms have been proposed for estimating motif counts [48, 61, 62, 72].

A.3 Additional Experiments

The following additional experiments are provided to complement the main empirical results. In particular, they examine whether the proposed methods remain effective and efficient when varying the temporal duration δ and the query temporal motif. These experiments are intended to verify that the empirical advantage of *AGA* and *HLA* is not limited to the default setting.

Exp-8: Running time of different temporal motif conductance algorithms with varying δ . Figure 11 shows the running time of *THSC*, *THLC*, *THPR*, *AGA*, and *HLA* with varying the temporal duration δ . As can be seen, we have (1) the consistent speed advantage of *HLA* stems from its purely local design. Rather than precomputing the weight $(W_{ef})_{uv}$ for all edges (u, v) in advance like other approaches, it calculates $(W_{ef})_{uv}$ on the fly when edge (u, v) is accessed. As a result, the runtime of *HLA* mainly depends on the locally explored region and the locally computed edge-focused weights, rather than on the overall graph scale. (Table 1). (2) Across all tested datasets and δ values, *AGA* consistently ranks as the second best performing method. Its efficiency is largely due to an innovative edge-centric weighted graph projection technique, which can be implemented effectively through the well-established temporal motif counting framework. In contrast, methods such as *THSC*, *THLC*, and *THPR* rely on exhaustively enumerating every temporal motif

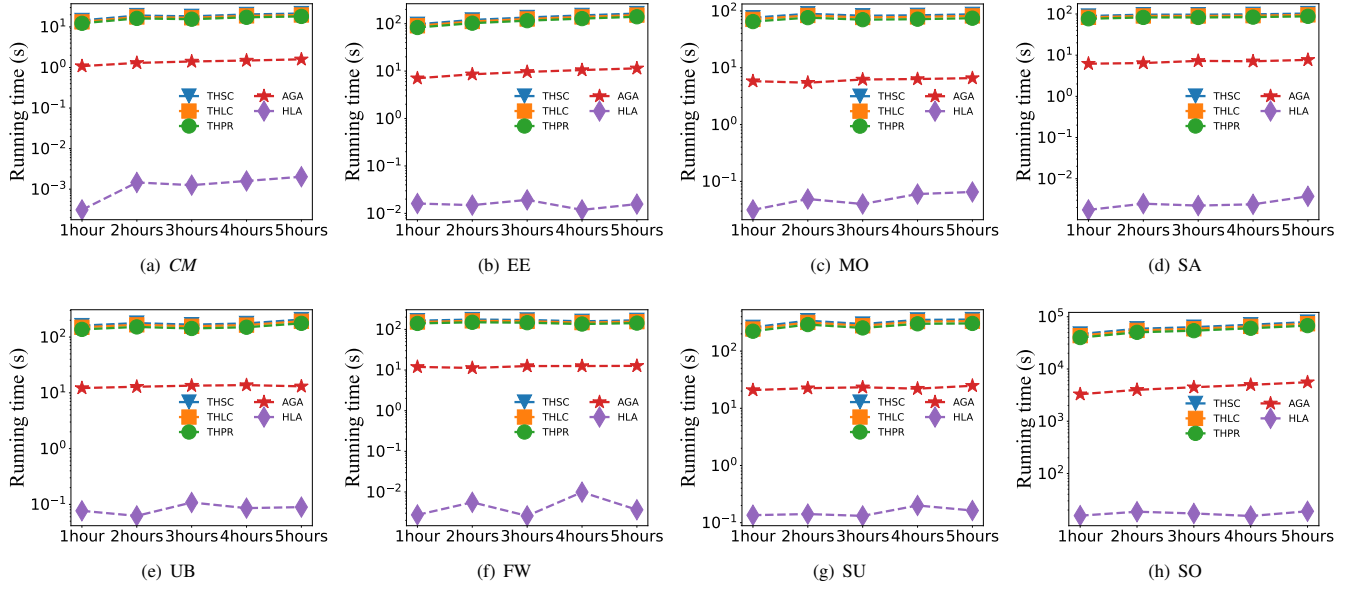


Figure 11: Running time of different temporal motif conductance algorithms with varying δ .

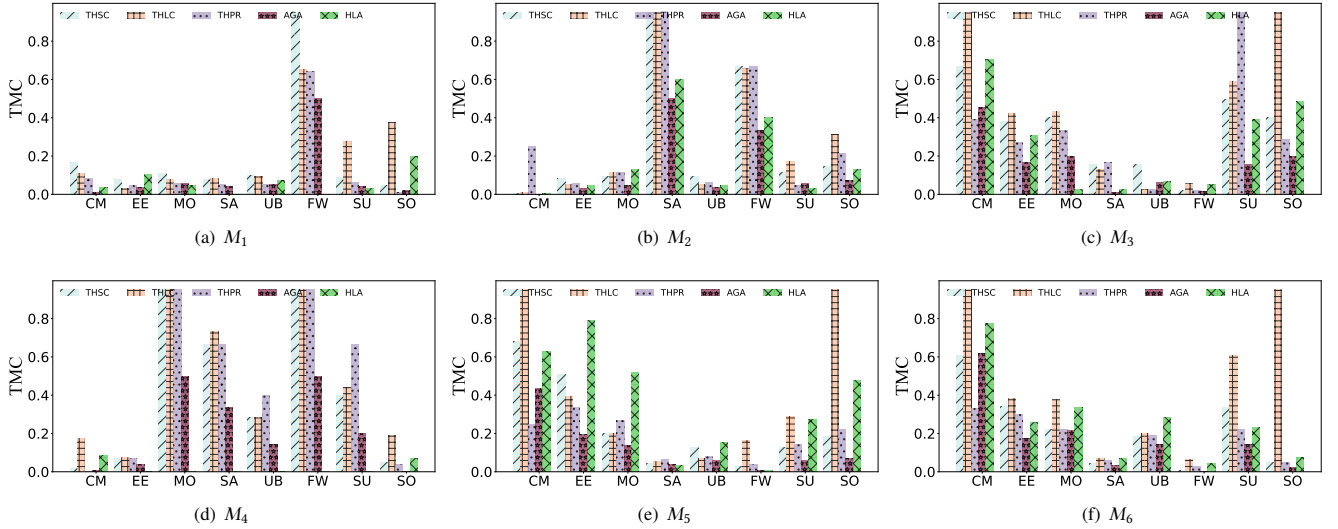


Figure 12: Effectiveness of different temporal motif conductance with varying temporal motif.

instance, leading to substantially longer execution times (Table 1). (3) All algorithms exhibit relatively stable performance as δ increases. This is because their time complexity mainly comes from the enumeration or counting of the temporal motif instance, but when δ increases within a relatively small range, it will not significantly affect their efficiency. In a nutshell, these results give preliminary evidence that the proposed solutions are highly efficient in practice.

Exp-9: Effectiveness of different temporal motif conductance algorithms with varying parameters. Based on the experimental results presented in Figures 12 and 13, the following key observations can be drawn: (1) The proposed algorithms, *AGA* and *HLA*,

demonstrate consistent superiority over existing baseline methods across a wide range of experimental configurations, including diverse temporal motif structures and varying temporal duration δ . Notably, despite the absence of theoretical guarantees, *HLA* achieves performance levels comparable to those of *AGA* in most scenarios. This suggests that the practical efficacy of *HLA* remains robust, highlighting its value in real-world applications where theoretical optimality may be secondary to empirical performance. (2) On average, the performance of our proposed *AGA* and *HLA* algorithms exhibit relative stability. This stability is evident across different datasets, temporal motifs, and temporal durations δ . In contrast,

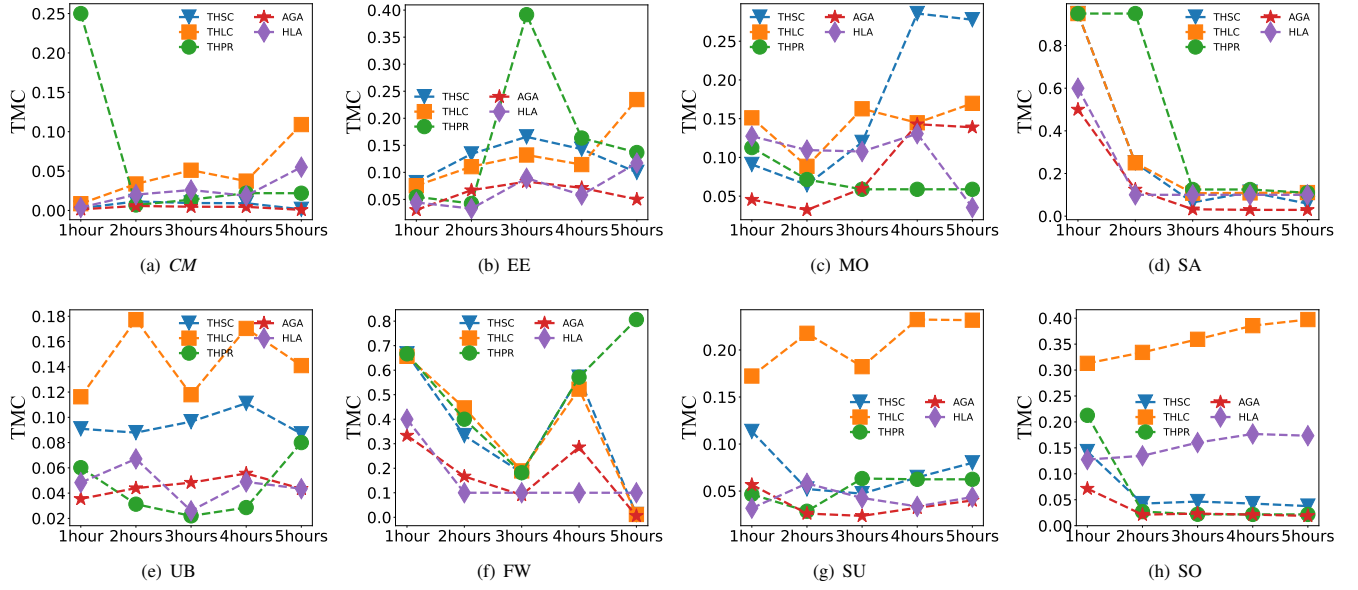


Figure 13: Effectiveness of different temporal motif conductance with varying δ .

the performance of other baseline methods displays considerable variability, which is highly dependent on the specific characteristics of the dataset, temporal motif, and temporal duration δ . These findings provide further empirical evidence supporting the notion that our algorithms are adept at identifying clusters of higher quality

compared to existing baseline methods when applied to real-world graph structures. This empirical validation aligns closely with the theoretical analysis presented in Table 1, reinforcing the theoretical underpinnings of our proposed methodologies.